```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.PetWorkCrew.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S15_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.PetWorkCrew.Boot
{
    /// &lt;summary&gt;
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
    static bool ConfigStep()
    {
```

```
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using Sy.PetWorkCrew.Game;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.PetWorkCrew.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new CopyLocaleService());   // T009 slice B: copy table = the
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
    public sealed class MockTimeService : ITimeService
```

```
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
    {
```

```
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// CopyLocaleService.cs
using System;
namespace Sy.PetWorkCrew.Game
{
    /// &lt;summary&gt;
    /// T009 slice B: the copy table as the game's locale service. Implements the
    /// shared Sy.Core ILocaleService face so the boot chain registers ONE text
    /// authority (replacing the key-fallback mock); lookups route CopyTable for
    /// both languages (dual-file doctrine: EN lives in CopyTable, ZH in
    /// CopyTableZh). The static CurrentLang is the face the pure static composers
    /// read - one SetLang call flips every copy face at once, and an unknown lang
    /// falls back to EN so no face can ever render blank.
    /// &lt;/summary&gt;
    public sealed class CopyLocaleService : Sy.Core.ILocaleService
    {
        /// &lt;summary&gt;Active copy language for every static composer (default EN).&lt;/summary&gt;
        public static string CurrentLang = CopyTable.LangEn;
        public void SetLang(string lang)
        {
            CurrentLang = (lang == CopyTable.LangEn || lang == CopyTable.LangZh)
                ? lang
                : CopyTable.LangEn;   // unknown lang: honest EN fallback, never a blank face
        }
        public string GetText(string key, params string[] args)
        {
            return CopyTable.Get(CurrentLang, key, args);
        }
        public Sy.Core.FontSet GetFontSet()
        {
            // procedural placeholder font triple (U020 two-state law: art batch swaps later)
            return new Sy.Core.FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
        }
    }
}
// CopyTable.cs
using System;
using System.Collections.Generic;
namespace Sy.PetWorkCrew.Game
{
    /// &lt;summary&gt;
    /// T009 copy table (requirement doc section 6, full batch): the single text
    /// authority for UI copy. English source strings live here (source-string
    /// English law); the Chinese localization lives in the separate CopyTableZh
    /// file (dual-file doctrine). Schema law: both files must cover the exact
    /// canonical key set with placeholder parity; structure moves bump
```

```
/// SchemaVersion. Views read copy only through Get (zero hardcoded UI text;
/// enforced by CopyTableCheck slice B once views are wired).
/// &lt;/summary&gt;
public static class CopyTable
{
    public const int SchemaVersion = 11;   // v2 = T011 ceremony batch (+9 keys); v3 = T012 honor wall batch (+10 ke
    public const string LangEn = "en";
    public const string LangZh = "zh";
    /// &lt;summary&gt;Canonical key set (schema face; mirrors requirement doc section 6).&lt;/summary&gt;
    public static readonly string[] Keys =
    {
        "welcome",
        "slogan",
        "dispatch_first",
        "offline_title",
        "offline_summary",
        "promote_toast",
        "postcard_from",
        "event_e01", "event_e02", "event_e03", "event_e04", "event_e05",
        "event_e06", "event_e07", "event_e08", "event_e09", "event_e10",
        "event_c01", "event_c02",                                  // U204 pick-two card titles
        "event_c01_a", "event_c01_b", "event_c02_a", "event_c02_b",   // U204 arm flavors (slice B view)
        "event_c00", "event_c00_a", "event_c00_b",                   // U204 C00 starter reach card (X1563: title +
        "zero_ad_achievement",
        "shop_stage_r1", "shop_stage_r2", "shop_stage_r3", "shop_stage_r4", "shop_stage_r5",
        "ceremony_stamp",
        "ceremony_unlock",
        "ceremony_crew",
        "ceremony_collect",
        "honor_wall",                     // T012 honor wall page title + button word
        "honor_unlocked",
        "honor_locked",
        "ach_first_wage",                 // T012 achievement names (zero-ad honor reuses zero_ad_achievement)
        "ach_full_crew",
        "ach_shop_star_full",
        "ach_postcard_collector",
        "ach_event_witness",
        "ach_global_corp",
        "ach_work_legend",
        "endgame_title",                   // T013 endgame finale (M10: full rank + all posts lit)
        "endgame_all_lit",
        "endgame_crew",
        "endgame_collect",
        "annual_title",                    // T013 annual report long image (M10 share hammer)
        "annual_stage",
        "annual_crew",
        "annual_albums",
        "annual_wage",
        "annual_share",
        "ngplus_run",                      // T013 slice D: NG+ view entry (the carried restart)
```

```
        "ngplus_fly",
        "ngplus_pick",                    // U204 slice 6: perk pick beat (three cultivation lanes)
        "ngplus_perk_crew",
        "ngplus_perk_album",
        "ngplus_perk_rep",
        "signin_title",                   // T026 slice B: daily sign-in popup (SYS retention beat)
        "signin_day",
        "signin_reward",
        "signin_claim",
        "signin_done",
        "signin_badge",
        "signin_x2",                      // T026-c2: sign-in x2 ad arm (the IAA 4th seat's popup button)
        "signin_x2_done",
    };
    static readonly Dictionary&lt;string, string&gt; En = BuildEn();
    static Dictionary&lt;string, string&gt; BuildEn()
    {
        var t = new Dictionary&lt;string, string&gt;();
        t["welcome"] = "Your crew is ready to work!";
        t["slogan"] = "You don't work. Cats &amp;amp; dogs work for you.";                        // pitch slogan (frozen
        t["dispatch_first"] = "First shift out! Wages incoming.";
        t["offline_title"] = "Welcome back, boss!";
        t["offline_summary"] = "While you were away, the crew kept working!";
        t["promote_toast"] = "Rank up! R{0}!";
        t["postcard_from"] = "From: your working crew";
        t["event_e01"] = "Corgi courier takes a tumble! Parcels everywhere!";
        t["event_e02"] = "Boss cat 'borrowed' one croissant. Zero regrets.";
        t["event_e03"] = "Lightning left the car in two tidy halves. Thumbs up!";
        t["event_e04"] = "A-wang fought the fire - and grilled sausages for the whole street!";
        t["event_e05"] = "Snowball's latte art turned into a dog. Served with a straight face.";
        t["event_e06"] = "Golden boy set the fish free. The cats staged a protest.";
        t["event_e07"] = "Doudou washed himself into a fluff ball and bowled over a bucket.";
        t["event_e08"] = "Dazhuang barked at the tree - fruit everywhere, surprise harvest!";
        t["event_e09"] = "Huajuan tipped a flower pot. Evidence: one bloom on the corgi's head.";
        t["event_e10"] = "Shadow knocked on the wrong door - and got a five-star tip!";
        t["event_c01"] = "A parcel mountain at the door! Haul it all, or pose for the photo first?";
        t["event_c01_a"] = "Haul the whole pile!";
        t["event_c01_b"] = "Pose for a photo first";
        t["event_c02"] = "The regular's farewell letter. Sprint the sunset route, or stay for tea?";
        t["event_c02_a"] = "Sprint the sunset route!";
        t["event_c02_b"] = "Stay for tea &amp;amp; a photo";
        t["event_c00"] = "A giant latte order! Chug the whole tray, or pose the latte-art photo first?";   // U204 X
        t["event_c00_a"] = "Chug the whole tray!";
        t["event_c00_b"] = "Pose the latte art first";
        t["zero_ad_achievement"] = "Zero-Ad Tycoon";                                          // U091 declaration achieve
        t["shop_stage_r1"] = "Alley Cat Cafe";                                               // M04 five-stage shop ladd
        t["shop_stage_r2"] = "Community Corner";
        t["shop_stage_r3"] = "Main-Street Flagship";
        t["shop_stage_r4"] = "City Chain";
        t["shop_stage_r5"] = "Global Pet Crew";
```

```
    t["ceremony_stamp"] = "STAMP! Officially R{0}!";                                // T011 stamp ceremony
    t["ceremony_unlock"] = "New posts unlocked: {0}";                               // promotion = ticket to ne
    t["ceremony_crew"] = "Crew for the new posts: {0}";
    t["ceremony_collect"] = "Back to work!";
    t["honor_wall"] = "Honor Wall";                              // T012 honor wall (SYS08)
    t["honor_unlocked"] = "Unlocked!";
    t["honor_locked"] = "Locked...";
    t["ach_first_wage"] = "First Paycheck!";
    t["ach_full_crew"] = "Ten Cats, Ten Dogs!";
    t["ach_shop_star_full"] = "First Post, Full Stars!";
    t["ach_postcard_collector"] = "Postcard Collector!";
    t["ach_event_witness"] = "Event Witness!";
    t["ach_global_corp"] = "Global Corporation!";
    t["ach_work_legend"] = "Work Legend!";
    t["endgame_title"] = "Global Pet Crew: World Stage Complete!";
    t["endgame_all_lit"] = "ALL LIT! {0}/10 posts strong!";
    t["endgame_crew"] = "The whole crew salutes you: {0}";
    t["endgame_collect"] = "Onward, boss!";
    t["annual_title"] = "Work Annual Report!";                      // T013 annual report (year-in-review brag
    t["annual_stage"] = "Stage: {0}";                               // stage name rides shop_stage_rN
    t["annual_crew"] = "Crew {0}/{1} · Posts lit {2}/{3}";
    t["annual_albums"] = "Postcards {0} · Events {1} · Honors {2}/{3}";
    t["annual_wage"] = "Wage beats: {0} · Stars: {1}/5";
    t["annual_share"] = "Share the year!";                          // finale overlay share button label
    t["ngplus_run"] = "New Run!";                                   // T013 slice D: finale new-run button (col
    t["ngplus_fly"] = "Crew &amp;amp; memories carried! Run {0} begins!";    // restart celebration fly ({0} = the r
    t["ngplus_pick"] = "Pick your keepsake for the new run!";       // U204 slice 6: perk pick prompt (the rest
    t["ngplus_perk_crew"] = "Old crew wisdom: wages x1.5!";         // crew lane (pack section 11, base-rate fa
    t["ngplus_perk_album"] = "Album memories: postcard bars x2!";   // album lane (pair progress face)
    t["ngplus_perk_rep"] = "Street rep: start at R1!";              // rep lane (starting rank face)
    t["signin_title"] = "Daily wage sign-in!";                     // T026 slice B: sign-in popup title
    t["signin_day"] = "Day {0} of 7 · streak {1}!";                 // cycle position + live streak
    t["signin_reward"] = "Next wage: +{0}c!";                       // the next claimable wage face
    t["signin_claim"] = "Sign in &amp;amp; collect!";                  // claim button label
    t["signin_done"] = "Signed today! Come back tomorrow!";         // claimed-state popup face
    t["signin_badge"] = "Sign-in!";                                 // home badge call face
    t["signin_x2"] = "Watch ad: x2 today's wage!";                 // T026-c2: x2 button call face (claimed, no
    t["signin_x2_done"] = "x2 collected! See you tomorrow!";        // x2 button done face (the day is doubled)
    return t;
}
public static bool HasKey(string key) =&amp;gt; En.ContainsKey(key);
/// &amp;lt;summary&amp;gt;Entry count of a language table (unknown lang = 0).&amp;lt;/summary&amp;gt;
public static int Count(string lang)
{
    if (lang == LangEn) return En.Count;
    if (lang == LangZh) return CopyTableZh.Count;
    return 0;
}
/// &amp;lt;summary&amp;gt;Raw lookup without formatting; missing key or lang = false.&amp;lt;/summary&amp;gt;
public static bool TryGetRaw(string lang, string key, out string value)
```

```
{
    if (lang == LangEn) return En.TryGetValue(key, out value);
    if (lang == LangZh) return CopyTableZh.TryGet(key, out value);
    value = null;
    return false;
}
/// &lt;summary&gt;Formatted lookup; missing key or lang falls back to the key itself.&lt;/summary&gt;
public static string Get(string lang, string key, params object[] args)
{
    string raw;
    if (!TryGetRaw(lang, key, out raw)) return key;
    if (args == null || args.Length == 0) return raw;
    try { return string.Format(raw, args); }
    catch (FormatException) { return raw; }
}
/// &lt;summary&gt;Placeholder count of a template (defensive: -1 on malformed braces).&lt;/summary&gt;
public static int PlaceholderCount(string template)
{
    if (string.IsNullOrEmpty(template)) return 0;
    int max = -1;
    for (int i = 0; i &lt; template.Length; i++)
    {
        if (template[i] != '{') continue;
        int close = template.IndexOf('}', i + 1);
        if (close &lt; 0) return -1;
        int n;
        if (!int.TryParse(template.Substring(i + 1, close - i - 1), out n)) return -1;
        if (n &gt; max) max = n;
        i = close;
    }
    return max + 1;
}
/// &lt;summary&gt;
/// Dual-file schema validation: exact key coverage both languages,
/// non-empty values, no stray keys, placeholder parity. Empty list = valid.
/// &lt;/summary&gt;
public static List&lt;string&gt; Validate()
{
    var errors = new List&lt;string&gt;();
    var zh = CopyTableZh.Snapshot();
    foreach (var k in Keys)
    {
        string enV;
        if (!En.TryGetValue(k, out enV)) errors.Add("missing EN key: " + k);
        else if (string.IsNullOrEmpty(enV)) errors.Add("empty EN value: " + k);
        string zhV;
        if (!zh.TryGetValue(k, out zhV)) errors.Add("missing ZH key: " + k);
        else if (string.IsNullOrEmpty(zhV)) errors.Add("empty ZH value: " + k);
        if (En.TryGetValue(k, out enV) &amp;&amp; zh.TryGetValue(k, out zhV))
        {
```

```
                int enP = PlaceholderCount(enV), zhP = PlaceholderCount(zhV);
                if (enP &lt; 0) errors.Add("malformed EN placeholder: " + k);
                if (zhP &lt; 0) errors.Add("malformed ZH placeholder: " + k);
                if (enP &gt;= 0 &amp;&amp; zhP &gt;= 0 &amp;&amp; enP != zhP)
                    errors.Add("placeholder parity broken: " + k + " (en=" + enP + " zh=" + zhP + ")");
            }
        }
        foreach (var k in En.Keys)
            if (Array.IndexOf(Keys, k) &lt; 0) errors.Add("stray EN key: " + k);
        foreach (var k in zh.Keys)
            if (Array.IndexOf(Keys, k) &lt; 0) errors.Add("stray ZH key: " + k);
        return errors;
    }
  }
}
// CopyTableZh.cs
using System.Collections.Generic;
namespace Sy.PetWorkCrew.Game
{
    /// &lt;summary&gt;
    /// T009 Chinese localization table (dual-file doctrine): the zh half of the
    /// copy schema. Key set must mirror CopyTable.Keys exactly; CopyTableCheck
    /// enforces coverage, non-empty values and placeholder parity. This file is
    /// localization data content (sanctioned by the T009 spec: source strings
    /// English in CopyTable, Chinese in this separate file).
    /// &lt;/summary&gt;
    internal static class CopyTableZh
    {
        static readonly Dictionary&lt;string, string&gt; Zh = Build();
        static Dictionary&lt;string, string&gt; Build()
        {
            var t = new Dictionary&lt;string, string&gt;();
            t["welcome"] = "         ";
            t["slogan"] = "           ";                    // pitch slogan (frozen)
            t["dispatch_first"] = "           ";
            t["offline_title"] = "        ";
            t["offline_summary"] = "                ";
            t["promote_toast"] = "   R{0} ";
            t["postcard_from"] = "        ";
            t["event_e01"] = "                  ";
            t["event_e02"] = "                ";
            t["event_e03"] = "                     ";
            t["event_e04"] = "                   ";
            t["event_e05"] = "                       ";
            t["event_e06"] = "                   ";
            t["event_e07"] = "                   ";
            t["event_e08"] = "           —           ";
            t["event_e09"] = "                     ";
            t["event_e10"] = "                        ";
            t["event_c01"] = "                      ";   // U204
```

```
t["event_c01_a"] = "     ";
t["event_c01_b"] = "     ";
t["event_c02"] = "                          ";
t["event_c02_a"] = "       ";
t["event_c02_b"] = "      ";
t["event_c00"] = "                          ";   // U204 X1563 C00             ·
t["event_c00_a"] = "      ";                                  //     0  +     30s·
t["event_c00_b"] = "      ";                                  // B  +4  +   + 2
t["zero_ad_achievement"] = "     ";                     // U091 declaration achievement
t["shop_stage_r1"] = "    ";                             // M04
t["shop_stage_r2"] = "    ";
t["shop_stage_r3"] = "       ";
t["shop_stage_r4"] = "    ";
t["shop_stage_r5"] = "        ";
t["ceremony_stamp"] = "        R{0} ";               // T011
t["ceremony_unlock"] = "      {0}";                   //   =
t["ceremony_crew"] = "          {0}";
t["ceremony_collect"] = "     ";
t["honor_wall"] = "   ";                                  // T012     SYS08
t["honor_unlocked"] = "    ";
t["honor_locked"] = "   ……";
t["ach_first_wage"] = "    ";
t["ach_full_crew"] = "    ";
t["ach_shop_star_full"] = "    ";
t["ach_postcard_collector"] = "      ";
t["ach_event_witness"] = "     ";
t["ach_global_corp"] = "    ";
t["ach_work_legend"] = "    ";
t["endgame_title"] = "                ";     // T013
t["endgame_all_lit"] = "     {0}/10      ";
t["endgame_crew"] = "          {0}";
t["endgame_collect"] = "        ";
t["annual_title"] = "     ";                                  // T013
t["annual_stage"] = "   {0}";                                  //       shop_stage_rN
t["annual_crew"] = "   {0}/{1} ·      {2}/{3}";
t["annual_albums"] = "    {0} ·    {1} ·    {2}/{3}";
t["annual_wage"] = "     {0} ·    {1}/5";
t["annual_share"] = "      ";                                  //
t["ngplus_run"] = "      ";                                   // T013   D
t["ngplus_fly"] = "           {0}      ";            //     fly {0} =
t["ngplus_pick"] = "           ";                      // U204   6 perk
t["ngplus_perk_crew"] = "         ×1.5 ";                 //         §11·
t["ngplus_perk_album"] = "           ×2 ";              //
t["ngplus_perk_rep"] = "        R1    ";                  //
t["signin_title"] = "       ";                                  // T026   b
t["signin_day"] = "    {0}    7  ·    {1}   ";         //    +
t["signin_reward"] = "     +{0}c ";                            //
t["signin_claim"] = "     ";                                   //
t["signin_done"] = "          ";                           //
t["signin_badge"] = "    ";                                        //
```

```
            t["signin_x2"] = "           ";                          // T026-c2 x2
            t["signin_x2_done"] = "          ";                       // x2
            return t;
        }
        public static int Count =&gt; Zh.Count;
        public static bool TryGet(string key, out string value) =&gt; Zh.TryGetValue(key, out value);
        /// &lt;summary&gt;Defensive copy for schema validation (never hands out the live table).&lt;/summary&gt;
        public static Dictionary&lt;string, string&gt; Snapshot() =&gt; new Dictionary&lt;string, string&gt;(Zh);
    }
}
// PetData.cs
using System;
namespace Sy.PetWorkCrew.Game
{
    /// &lt;summary&gt;
    /// Frozen schema mirror v1 of the G15 numeric truth pack (G15_   .md = single
    /// source of numbers). Law: structure moves require SchemaVersion bump; values may
    /// be re-tuned by the bot sweep gate (pack re-freezes, this mirror syncs, checks
    /// re-run). Sections below map 1:1 to the pack: jobs(1) / promote gates(2) /
    /// star law(3) / events(5) / recruit roster(4) / pace anchors(9). Offline(6) /
    /// adcap(7) / postcard(8) tables land with their own task slices (T007-T008);
    /// this mirror stays frozen meanwhile. Section 1 law 3 job_coffee degeneration (open cost 0
    /// -&gt; upgrade cost 0 forever) was disposed by the T004 bot-sweep refreeze: frozen
    /// coffee upgrade base 250 * level (sweep grid [200..600] step 50, first-upgrade
    /// payback anchor closest to the 60s decision cadence; JobPoolCheck proves the
    /// freeze in-check).
    /// &lt;/summary&gt;
    public static class PetData
    {
        public const int SchemaVersion = 8;   // v1 = T002 mirror; v2 = T003 pet dex; v3 = T006 event table; v4 = T008 p
        // ----- pack section 9: pace anchors (red line 11 assertion source) -----
        public const float FirstDispatchFeedbackMaxSec = 1.0f;  // coin ding three-sync bound
        public const float FirstEventCardMaxSec = 10.0f;        // E01 corgi tumble beat (T006)
        public const float P1TargetSec = 90f;                    // first job + first wage
        public const float P50TargetSec = 240f;                  // second job + first postcard
        // ----- pack section 1: job rate table (10 jobs; free first job = P1 anchor) -----
        public const int JobCount = 10;
        public const int JobCoffee = 0;
        public const int JobCourier = 1;
        public const int JobCarwash = 2;
        public const int JobFish = 3;
        public const int JobFlower = 4;
        public const int JobBakery = 5;
        public const int JobBook = 6;
        public const int JobGarage = 7;
        public const int JobOrchard = 8;
        public const int JobFire = 9;
        public sealed class JobDef
        {
            public string Id;
```

```
    public int UnlockRank;        // shop rank index (R1=0 .. R5=4) required to open
    public float BaseRatePerSec;   // coins per second at level 1
    public double OpenCost;        // first-open coin cost
}
public static readonly JobDef[] Jobs =
{
    new JobDef { Id = "job_coffee",   UnlockRank = 0, BaseRatePerSec = 40f,  OpenCost = 0 },
    new JobDef { Id = "job_courier",  UnlockRank = 1, BaseRatePerSec = 60f,  OpenCost = 300 },
    new JobDef { Id = "job_carwash",  UnlockRank = 1, BaseRatePerSec = 55f,  OpenCost = 280 },
    new JobDef { Id = "job_fish",     UnlockRank = 2, BaseRatePerSec = 90f,  OpenCost = 800 },
    new JobDef { Id = "job_flower",   UnlockRank = 2, BaseRatePerSec = 85f,  OpenCost = 750 },
    new JobDef { Id = "job_bakery",   UnlockRank = 3, BaseRatePerSec = 130f, OpenCost = 1800 },
    new JobDef { Id = "job_book",     UnlockRank = 3, BaseRatePerSec = 120f, OpenCost = 1600 },
    // T004 payability refreeze: garage/orchard were pack-listed R5 which deadlocked
    // the R4-&gt;R5 gate (8 jobs required vs 7 openable at R4); re-frozen to R4 per
    // requirement doc section 6 (both are tier-4 jobs) - JobPoolCheck L12 proves it.
    new JobDef { Id = "job_garage",   UnlockRank = 3, BaseRatePerSec = 180f, OpenCost = 3600 },
    new JobDef { Id = "job_orchard",  UnlockRank = 3, BaseRatePerSec = 170f, OpenCost = 3200 },
    new JobDef { Id = "job_fire",     UnlockRank = 4, BaseRatePerSec = 250f, OpenCost = 8000 },
};
/// &lt;summary&gt;Pack law 2: log soft cap - level output = base * (1 + 0.25 * ln(level + 1)).&lt;/summary&gt;
public static double JobRatePerSec(int jobId, int level)
{
    return Jobs[jobId].BaseRatePerSec * (1.0 + 0.25 * Math.Log(level + 1.0));
}
/// &lt;summary&gt;T004 bot-sweep refreeze: frozen coffee upgrade base (see JobUpgradeCost).&lt;/summary&gt;
public const double CoffeeUpgradeCostBase = 250;
/// &lt;summary&gt;Pack law 3: linear job price - upgrade cost = first-open cost * level (no
/// exponent). T004 bot-sweep refreeze: job_coffee opens free (P1 anchor) which left
/// this law degenerate at zero forever; the frozen base 250 * level wins the sweep
/// grid [200..600] step 50 on the first-upgrade payback anchor (250 / 4.0546 =
/// 61.7s, closest to the 60s core-loop decision cadence).&lt;/summary&gt;
public static double JobUpgradeCost(int jobId, int level)
{
    if (jobId == JobCoffee) return CoffeeUpgradeCostBase * level;
    return Jobs[jobId].OpenCost * level;
}
/// &lt;summary&gt;R160 invest cap: upgrades stop at this level - the coins must
/// flow to the next job instead of averaging into one (anti "spread
/// resources evenly" misclick; capped refusal is zero-state-change).&lt;/summary&gt;
public const int JobMaxLevel = 30;
// ----- pack section 2: five-stage promote gates (monotonic triple thresholds) -----
public const int RankCount = 5;
/// &lt;summary&gt;Gate from rank i to rank i+1: open jobs / recruited pets (dex) / star level.&lt;/summary&gt;
public static readonly int[] GateJobsAtLeast = { 1, 3, 5, 8 };
public static readonly int[] GateDexAtLeast = { 2, 4, 7, 10 };
public static readonly int[] GateStarsAtLeast = { 1, 2, 3, 4 };
// ----- pack section 3: reputation star law (no decay, no check-in, no contest) -----
public const float StarDustPerMinPerPet = 1.0f;   // per on-duty pet per minute
public const float StarDustDexBonusCoef = 0.05f;  // multiplier (1 + 0.05 * dex count)
```

```
public const float StarUpCostBase = 100f;         // star up cost = base * (star level + 1)
public const int StarMax = 5;
// ----- pack section 5: comedy event table (T006 slice A data face; SYS04 EventDef{ID,job,type,fx,collect}) ---
// Law (pack section 5): online cadence 180-300s rolls among on-duty pets' jobs,
// each event settles once per occurrence as a star-dust correction (+/- 5..10)
// plus an event-album collect bit (SYS08). Batch values are all positive:
// comedy never punishes (pack section 4 law 3 - flip-a-pet and quirks are
// comedy beats, zero numeric penalty, R73 red line). E01 doubles as the
// scripted red-line-11 first-laugh card (see PetSim.FirstEventCardAtSec).
public const int EventCount = 10;  // full pool E01-E10 (T010 second batch; requirement doc section 6)
public enum EventKind { Comedy = 0, Heartwarming = 1 }
public sealed class EventDef
{
    public string Id;
    public int Job;            // job index the event belongs to (requirement doc section 6 event table)
    public EventKind Kind;
    public string Fx;           // view fx stem (slice B pop layer consumes)
    public double StarDelta;    // star dust correction, pack window |delta| in [5,10]
    public bool Collect;       // event album (SYS08) collectible
    public bool Choice;           // U204: pick-two card (auto pool stays RollCadenceEvent-excluded)
    public double ChoiceAStar;    // U204 arm A: big stardust now (short value)
    public double ChoiceBStar;    // U204 arm B: smaller stardust ...
    public bool ChoiceBCollect;   // ... plus event-album bit (long value; no dominant arm)
    public int ChoiceFlag;        // story flag the B arm sets (-1 = none)
    public bool ChoiceRiskArm;    // U204 FQ slice b: arm A is the declared risk arm (data face, non-hardcoded)
    public int ReqFlag;           // story flag required for eligibility (-1 = always)
}
// T010 second batch E04-E10: heartwarming beats mixed into the tumble-comedy
// pool (pack section 5 / R73 warm-detail anchor) - E04 and E10 carry the warm
// endings (fire rescue + sausage party, wrong-door five-star tip), the rest
// stay comedy. All deltas positive (comedy never punishes); jobs mirror the
// doc table: fire/coffee/fish/carwash/orchard/flower + E10 double-binds courier.
public static readonly EventDef[] Events =
{
    new EventDef { Id = "event_e01", Job = JobCourier, Kind = EventKind.Comedy,        Fx = "fx_parcel_tumble",
    new EventDef { Id = "event_e02", Job = JobBakery,  Kind = EventKind.Comedy,        Fx = "fx_cam_feign_sleep"
    new EventDef { Id = "event_e03", Job = JobGarage,  Kind = EventKind.Comedy,        Fx = "fx_thumbs_up_wreck"
    new EventDef { Id = "event_e04", Job = JobFire,    Kind = EventKind.Heartwarming, Fx = "fx_sausage_rescue",
    new EventDef { Id = "event_e05", Job = JobCoffee,  Kind = EventKind.Comedy,        Fx = "fx_dog_latte",
    new EventDef { Id = "event_e06", Job = JobFish,     Kind = EventKind.Comedy,        Fx = "fx_fish_release",
    new EventDef { Id = "event_e07", Job = JobCarwash, Kind = EventKind.Comedy,        Fx = "fx_furball_rollover
    new EventDef { Id = "event_e08", Job = JobOrchard,  Kind = EventKind.Comedy,        Fx = "fx_roar_harvest",
    new EventDef { Id = "event_e09", Job = JobFlower,  Kind = EventKind.Comedy,        Fx = "fx_frame_the_corgi"
    new EventDef { Id = "event_e10", Job = JobCourier, Kind = EventKind.Heartwarming, Fx = "fx_wrong_door_fivest
};
// ----- U204 choice cards (R161 spectrum 2026-09-25: Reigns pick-two x cozy
// variant; data layer slice A - the View ships the two-button card in slice B) -----
// Spectrum laws: a choice weights the tap with consequences (meaning law);
// cozy arms stay dual-positive - the ditch is "earn less", never lose (R73 red
// line). EXCEPTION (U204 FQ slice b, X1554 freeze): the DECLARED risk arm
```

```
// (ChoiceRiskArm, data face) carries zero stardust and plants a RiskPauseSec
// wage pause on the card's source job - the honest loss face that keeps the
// strategy seat from a structural 1.0 win rate (AntiDumb F1). The choice
// cadence rides once per 3-5 AUTO-card resolves (anti decision-fatigue
// window), and the B arm on C01 plants the flag that unlocks C02 (callback arc
// -&gt; event-album linkage: choice -&gt; unlock -&gt; album).
public const int ChoiceCount = 3;               // C01 + C02 (flag-arc pair) + C00 (starter-post reach card, X15
public const int ChoiceFlagCount = 3;           // story flags the B arms can plant (flag 2 = C00 starter arc bi
public const int ChoiceCadenceMinResolves = 1;  // choice window = 1-2 auto resolves (X1563 refreeze:
public const int ChoiceCadenceMaxResolves = 2;  // event pacing 180-300s made the 3-5 window = one choice per
// 9-25min, starving a fresh 240s sitting to zero cards - the freeze sec2 math assumed one card
// per sitting (12.5% erosion cap); 1-2 restores that density, R161 fatigue still held by the
// 180-300s event pacing itself (one decision surface per 3-10min, one at a time)
/// &lt;summary&gt;U204 FQ slice b (X1554 freeze): risk-arm wage-pause window in
/// seconds (one card in a 240s sitting = 12.5% erosion cap).&lt;/summary&gt;
public const double RiskPauseSec = 30.0;
/// &lt;summary&gt;Ships true since slice b-2: the two-button choice card is live and
/// every long-walk bot drive settles pending choices with player-semantic
/// arm taps (one decision at a time, R161 reading-and-deciding law).&lt;/summary&gt;
public static bool ChoiceViewLive = true;
public static readonly EventDef[] ChoiceEvents =
{
    // C01 comedy: the courier cat meets a parcel pile - haul it all (arm A,
    // DECLARED RISK ARM: the slapstick tumble leaves the courier sulking -
    // zero stardust + RiskPauseSec wage pause, X1554 freeze) vs pose for a
    // photo first (arm B, +stardust, album bit + plants flag 0 that unlocks C02).
    new EventDef { Id = "event_c01", Job = JobCourier, Kind = EventKind.Comedy,
        Fx = "fx_parcel_pile_picktwo", StarDelta = 8, Collect = false,
        Choice = true, ChoiceAStar = 0, ChoiceBStar = 4, ChoiceBCollect = true,
        ChoiceFlag = 0, ChoiceRiskArm = true, ReqFlag = -1 },
    // C02 heartwarming (flag-gated arc): the regular's farewell letter run -
    // sprint the sunset route (arm A, DECLARED RISK ARM: zero stardust +
    // RiskPauseSec wage pause) vs stay for the tea-and-photo goodbye (arm B,
    // +stardust, album bit + flag 1). Courier double-bind mirrors the
    // E01/E10 precedent (the arc stays provable with one staffed job).
    new EventDef { Id = "event_c02", Job = JobCourier, Kind = EventKind.Heartwarming,
        Fx = "fx_farewell_letter_picktwo", StarDelta = 8, Collect = false,
        Choice = true, ChoiceAStar = 0, ChoiceBStar = 4, ChoiceBCollect = true,
        ChoiceFlag = 1, ChoiceRiskArm = true, ReqFlag = 0 },
    // C00 comedy reach card (X1563): rides the STARTER coffee post so the
    // declared risk arm is reachable inside a fresh 240s sitting (X1554
    // freeze sec5 anticipated the courier rank1 gate keeps C01/C02 beyond
    // the window - this card closes the reach block without touching the
    // rank ladder). Same frozen risk-arm semantics: arm A = zero stardust +
    // RiskPauseSec wage pause on the coffee post; arm B = +4 stardust,
    // album bit, plants flag 2 (starter arc bit, no gate reads it).
    new EventDef { Id = "event_c00", Job = JobCoffee, Kind = EventKind.Comedy,
        Fx = "fx_latte_chug_picktwo", StarDelta = 8, Collect = false,
        Choice = true, ChoiceAStar = 0, ChoiceBStar = 4, ChoiceBCollect = true,
        ChoiceFlag = 2, ChoiceRiskArm = true, ReqFlag = -1 },
```

```
};
// ----- pack section 4: recruit roster (coin direct-buy, zero gacha; monotonic cost chain) -----
public const int PetCount = 12;
public const int PetOrange = 0;   // starter cat, granted free at boot (pack: open gift)
public const int PetAwang = 11;  // fire-station end-game companion pet (pack section 4)
public static readonly string[] PetIds =
{
    "pet_orange", "pet_snowball", "pet_corgi", "pet_goldie", "pet_bean",
    "pet_sesame", "pet_swirl", "pet_pudding", "pet_shadow", "pet_brawn",
    "pet_flash", "pet_awang",
};
public static readonly double[] RecruitCosts =
{
    0, 200, 350, 600, 850, 1200, 1500, 2200, 3200, 4500, 6500, 9000,
};
// ----- pack section 4 extension (T003 slice A): pet dex table -----
// Mirror of the requirement doc pet table (6 cats + 6 dogs) mapped onto the
// pack law 2 passive closed set {revenue_pct, speed_pct, star_pct, quirk}.
// Index order is contract-bound to PetIds / RecruitCosts above. Flavor text
// lives in the doc; the mapping judgment (output/efficiency/foam/carry -&gt;
// RevenuePct, speed -&gt; SpeedPct, reputation -&gt; StarPct) is recorded here so
// the bot-sweep refreeze channel can re-tune values without re-deriving it.
// Structure move vs mirror v1 =&gt; SchemaVersion bumped to 2.
public enum PetSpecies { Cat = 0, Dog = 1 }
public enum PassiveKind { RevenuePct = 0, SpeedPct = 1, StarPct = 2 }
public sealed class PetDef
{
    public string Id;
    public string NameKey;       // localization table key (S2 copy batch)
    public PetSpecies Species;
    public int AffinityJob;      // job index this pet is drawn to (job table)
    public PassiveKind Passive;
    public double PassiveValue;   // e.g. 0.10 = +10%
    public string QuirkId;       // comedy quirk event key (T006 engine feeds on this)
}
public static readonly PetDef[] PetDex =
{
    new PetDef { Id = "pet_orange",   NameKey = "pet_name_orange",   Species = PetSpecies.Cat,  AffinityJob = Jo
    new PetDef { Id = "pet_snowball", NameKey = "pet_name_snowball", Species = PetSpecies.Cat,  AffinityJob = Jo
    new PetDef { Id = "pet_corgi",    NameKey = "pet_name_corgi",    Species = PetSpecies.Dog,  AffinityJob = Jo
    new PetDef { Id = "pet_goldie",    NameKey = "pet_name_goldie",   Species = PetSpecies.Dog,  AffinityJob = J
    new PetDef { Id = "pet_bean",      NameKey = "pet_name_bean",     Species = PetSpecies.Dog,  AffinityJob = J
    new PetDef { Id = "pet_sesame",    NameKey = "pet_name_sesame",   Species = PetSpecies.Cat,  AffinityJob = J
    new PetDef { Id = "pet_swirl",     NameKey = "pet_name_swirl",    Species = PetSpecies.Cat,  AffinityJob = J
    new PetDef { Id = "pet_pudding",   NameKey = "pet_name_pudding",  Species = PetSpecies.Cat,  AffinityJob = J
    new PetDef { Id = "pet_shadow",    NameKey = "pet_name_shadow",   Species = PetSpecies.Cat,  AffinityJob = J
    new PetDef { Id = "pet_brawn",     NameKey = "pet_name_brawn",    Species = PetSpecies.Dog,  AffinityJob = J
    new PetDef { Id = "pet_flash",     NameKey = "pet_name_flash",    Species = PetSpecies.Dog,  AffinityJob = J
    new PetDef { Id = "pet_awang",     NameKey = "pet_name_awang",    Species = PetSpecies.Dog,  AffinityJob = J
};
```

```
// ----- U204 slice 3 (dispatch strategy): affinity match wage law -----
// R160 (talent x post match spectrum): dispatch is a real decision - a pet
// working its AffinityJob earns +25% wage beats. Positive feedback only:
// a mismatch earns the exact base rate, never a penalty (C070 cozy law).
// The bonus is a base-rate face like the NG+ crew perk (never an ad gain),
// so the ad x2 red line and every ratio/isomorphism law stay exact.
public const double AffinityWageCoef = 1.25;
/// &lt;summary&gt;True when petId's dex AffinityJob equals jobId (the match pays
/// AffinityWageCoef on that job's wage beats; a mismatch pays exact base).&lt;/summary&gt;
public static bool IsAffinityMatch(int petId, int jobId)
{
    return petId &gt;= 0 &amp;&amp; petId &lt; PetCount &amp;&amp; PetDex[petId].AffinityJob == jobId;
}
// ----- pack section 8: wage postcards (T008 slice A; SYS08/M08) -----
// Pack law (numeric pack section 8): the postcard album is a PROGRESS-BAR
// collection - work time accumulates per (pet, job) pair, zero probability,
// guaranteed payout at a full bar (R72 counter-evidence: "letters were pure
// luck"). One collectible postcard settles per pair per full 4h work bar;
// online beats and offline credited spans both count (true idle game).
// Collection capacity starts at 12 pets x 10 jobs = 120 pair cards.
public const double PostcardWorkSecPerCard = 4.0 * 3600.0;   // one card per full 4h work bar
public const int PostcardPairCount = PetCount * JobCount;    // 120 collectible pair cards
public const int PostcardAlbumCapacity = PostcardPairCount;  // pack: capacity 120 starting (grows with content)
public enum PostcardKind { Wage = 0, Away = 1 }
public sealed class PostcardDef
{
    public string Id;         // pc_&lt;pet&gt;_&lt;job&gt; - album + share key face
    public int Pet;
    public int Job;
    public string FlavorKey;  // localization key (slice B card face; flavor text lives in the doc template tabl
}
static PostcardDef[] BuildPostcardTable()
{
    var t = new PostcardDef[PostcardPairCount];
    int k = 0;
    for (int p = 0; p &lt; PetCount; p++)
        for (int j = 0; j &lt; JobCount; j++)
            t[k++] = new PostcardDef
            {
                Id = "pc_" + PetIds[p].Substring(4) + "_" + Jobs[j].Id.Substring(4),
                Pet = p,
                Job = j,
                FlavorKey = "pc_flavor_" + PetIds[p] + "_" + Jobs[j].Id,
            };
    return t;
}
public static readonly PostcardDef[] Postcards = BuildPostcardTable();
// ----- requirement doc section 6 achievement table (T012; SYS08 honor wall) -----
// Eight honors, one per doc row. Conditions map 1:1 onto the implemented
// systems (each judgment recorded here so the bot-sweep refreeze channel
```

```
// can re-tune without re-deriving):
//   FirstWage        = first wage beat credited (SYS01; P1 anchor ~90s)
//   FullCrew         = all 12 pets recruited (SYS03 dex full; flavor name
//                      "ten cats, ten dogs" rides the roster of 6 + 6)
//   ShopStarFull     = shop reputation star reaches StarMax (SYS06 is the
//                      shop-wide star face - "first post, full stars" honors
//                      the alley cafe filling its star card)
//   PostcardCollector= PostcardCollectorTarget cards journaled (SYS08;
//                      collector tier = a dozen cards, progress-bar law)
//   EventWitness     = every pool event collected in the event album (SYS04/SYS08)
//   GlobalCorp       = rank ladder top R5 reached (SYS05 top-rank stage)
//   ZeroAdClear      = U091 declaration honor: R5 reached with ZERO ads
//                      ever watched (core progress is fully ad-free;
//                      evt_zero_ad_clear telemetry rides the unlock)
//   WorkLegend       = all of the above unlocked (the legend closes the wall)
// Structure move vs mirror v4 =&gt; SchemaVersion bumped to 5.
public const int AchievementCount = 8;
public const int AchFirstWage = 0;
public const int AchFullCrew = 1;
public const int AchShopStarFull = 2;
public const int AchPostcardCollector = 3;
public const int AchEventWitness = 4;
public const int AchGlobalCorp = 5;
public const int AchZeroAdClear = 6;
public const int AchWorkLegend = 7;
public const int PostcardCollectorTarget = 12;   // collector tier: a dozen cards
public sealed class AchievementDef
{
    public string Id;        // schema id (achievement engine face)
    public string NameKey;   // localization table key (honor wall name face)
}
// The zero-ad honor reuses the frozen zero_ad_achievement copy key
// (requirement doc section 6 keeps it in the T009 key set already).
public static readonly AchievementDef[] Achievements =
{
    new AchievementDef { Id = "ach_first_wage",         NameKey = "ach_first_wage" },
    new AchievementDef { Id = "ach_full_crew",          NameKey = "ach_full_crew" },
    new AchievementDef { Id = "ach_shop_star_full",     NameKey = "ach_shop_star_full" },
    new AchievementDef { Id = "ach_postcard_collector", NameKey = "ach_postcard_collector" },
    new AchievementDef { Id = "ach_event_witness",      NameKey = "ach_event_witness" },
    new AchievementDef { Id = "ach_global_corp",        NameKey = "ach_global_corp" },
    new AchievementDef { Id = "ach_zero_ad_clear",      NameKey = "zero_ad_achievement" },
    new AchievementDef { Id = "ach_work_legend",         NameKey = "ach_work_legend" },
};
// ----- T013 slice C: NG+ inheritance coefficients (pack section 11 freeze
// slot - machine-set, NumericsCheck T014 asserts them in-table). -----
/// &lt;summary&gt;
/// NG+ dex album inheritance coefficient. Machine-set 1.0 = full carry: the
/// pet dex (and with it the event album) crosses the restart whole - a
/// restart never wipes a collection page (R72 collection-guarantee /
```

```
/// R73 save-safety anchors; U091 experience-first: the rebuild arc is the
/// economy, never the collection). The coefficient stays the tuning dial;
/// the value encodes the zero-collection-punishment law.
/// &lt;/summary&gt;
public const double DexInheritCoef = 1.0;
/// &lt;summary&gt;
/// NG+ postcard inheritance coefficient. Machine-set 1.0 = the pair album,
/// the bounded journal and the honor wall carry in full (mail is memories;
/// honors are lifetime - the zero-ad honor never re-locks). Pair work
/// progress bars reset with the economy so the fresh arc replays clean.
/// &lt;/summary&gt;
public const double PostcardInheritCoef = 1.0;
// ----- U204 flow-optimize depth slice: NG+ inheritance cultivation
// perks (pack section 11 extension). One perk is chosen per restart and
// shapes ONLY the new run - the carried-collection law (both inheritance
// coefficients above stay 1.0) and the reset law are untouched, so the
// unperked restart stays byte-identical to the slice C/D behavior.
// Depth target: the audit verdict "coefficient 1.0 = no-feel
// inheritance" - a chosen cultivation lane makes the second run PLAY
// differently (wage lane / collection lane / reputation lane), the
// replay arc becomes a decision instead of a repeat. -----
public enum NgPlusPerk { None = 0, Crew = 1, Album = 2, Rep = 3 }
public sealed class NgPlusPerkDef
{
    public NgPlusPerk Perk;
    public string Id;          // schema id (the choice-face copy lands with the view slice)
    public double WageCoef;    // new-run wage beat multiplier (1.0 = no effect)
    public double PairBarCoef; // new-run pair postcard bar fill multiplier (1.0 = no effect)
    public int StartRank;      // new-run starting rank (0 = fresh ladder)
}
// Machine-set per the pack refreeze channel: crew wisdom = +50% wages
// (matches the affinity-bonus scale the depth audit recommended, inside
// the ad x2 red line because it is a base-rate face, never an ad gain);
// photo memories = pair bars fill at 2x (the album lane replays the
// collection arc in half the work time); star reputation = the new run
// opens at rank 1 (the neighborhood remembers - the R2 gate is already
// half-earned, never a free ride: jobs/dex/star gates still apply).
public static readonly NgPlusPerkDef[] NgPlusPerks =
{
    new NgPlusPerkDef { Perk = NgPlusPerk.None,  Id = "ngplus_perk_none",  WageCoef = 1.0, PairBarCoef = 1.0, St
    new NgPlusPerkDef { Perk = NgPlusPerk.Crew,  Id = "ngplus_perk_crew",  WageCoef = 1.5, PairBarCoef = 1.0, St
    new NgPlusPerkDef { Perk = NgPlusPerk.Album, Id = "ngplus_perk_album", WageCoef = 1.0, PairBarCoef = 2.0, St
    new NgPlusPerkDef { Perk = NgPlusPerk.Rep,   Id = "ngplus_perk_rep",   WageCoef = 1.0, PairBarCoef = 1.0, St
};
/// &lt;summary&gt;Perk table lookup; unknown ids fold to the None def (the
/// sim layer refuses unknown ids outright - zero-state-change law).&lt;/summary&gt;
public static NgPlusPerkDef NgPlusPerkDefOf(NgPlusPerk perk)
{
    for (int i = 0; i &lt; NgPlusPerks.Length; i++)
        if (NgPlusPerks[i].Perk == perk) return NgPlusPerks[i];
```

```
            return NgPlusPerks[0];
        }
        // ----- pack section 12: sign-in retention law (T026 slice A; R162 check-in
        // spectrum; M11 retention-segment face). Mirror of the numeric pack section
        // 12 freeze: one claim per wall-clock day, streak counts consecutive days,
        // a gap soft-resets the streak to day 1 (comedy zero-penalty face - the
        // cozy law never punishes a missed day, R73 red line), and the reward walks
        // a fixed 7-day coin cycle anchored 60..300 (monotonic ramp with a big-day
        // payout on day 7). The day domain is WALL-CLOCK fed by the caller (X995
        // law: an uptime-domain day is blind across sessions) - the sim never
        // reads a clock itself. Cross-session persistence rides the save-system
        // batch (honest note: G15 ships no global save yet - the whole sim is
        // session-scoped today; this slice owns the law + op, the View + persistence
        // land with slices b/c and the save-system task). Structure move vs mirror
        // v7 =&gt; SchemaVersion bumped to 8. -----
        public const int SignInCycleDays = 7;
        public const double SignInRewardDay1 = 60;    // pack section 12 anchor row (7-day cycle)
        public const double SignInRewardDay7 = 300;  // big-day payout anchor
        /// &lt;summary&gt;Fixed 7-day coin cycle (pack section 12 freeze: 60/80/100/120/150/200/300).&lt;/summary&gt;
        public static readonly double[] SignInRewards = { 60, 80, 100, 120, 150, 200, 300 };
        /// &lt;summary&gt;Reward for a given streak day: day 1 -&gt; 60 ... day 7 -&gt; 300, day 8 wraps to day 1.&lt;/
        public static double SignInRewardForStreak(int streak)
        {
            int idx = ((streak - 1) % SignInCycleDays + SignInCycleDays) % SignInCycleDays;
            return SignInRewards[idx];
        }
    }
}
// PetMainHost.cs
using System.Globalization;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.PetWorkCrew.Game
{
    /// &lt;summary&gt;
    /// Three-sync feedback contract: one wage beat answers in the SAME frame with
    /// all three faces at once - the ding sound, the readout pulse and the flying
    /// number (red line 11: first dispatch feedback &amp;lt;= 1.0s, "sound/art/number
    /// three-sync", requirement doc section 1 beat table). The composer is a pure
    /// static function so the Redline11Check gate can assert the contract without
    /// a scene; the host applies the three faces the same frame the beat fires.
    /// &lt;/summary&gt;
    public readonly struct CoinFeedback
    {
        public readonly string SoundKey;    // audio stem key (placeholder ding until the G15 audio batch lands)
        public readonly float PulseScale;  // readout pop scale (&gt;1 means the pulse face fires)
        public readonly string NumberText;  // flying "+x.xx" wage number
        public readonly int JobId;
        public readonly float BeatTime;
        public CoinFeedback(string soundKey, float pulseScale, string numberText, int jobId, float beatTime)
```

```
        {
            SoundKey = soundKey;
            PulseScale = pulseScale;
            NumberText = numberText;
            JobId = jobId;
            BeatTime = beatTime;
        }
    }
    /// &lt;summary&gt;
    /// Work-theater opening face for a dispatch (T002 "the work theater opens on
    /// first dispatch"): the pet appears at its job post the same frame the
    /// assignment lands. Pure contract half for the gate; the host owns the visual.
    /// &lt;/summary&gt;
    public readonly struct TheaterOpen
    {
        public readonly string JobLabel;
        public readonly string PetLabel;
        public readonly int JobId;
        public readonly int PetId;
        public TheaterOpen(string jobLabel, string petLabel, int jobId, int petId)
        {
            JobLabel = jobLabel;
            PetLabel = petLabel;
            JobId = jobId;
            PetId = petId;
        }
    }
    /// &lt;summary&gt;
    /// T002 slice B-1b: main-scene runtime host. The scene file stays a shell
    /// (camera + canvas); this host builds the wage HUD procedurally (U020
    /// D-grade placeholder faces - the art batch swaps in catalog sprites later
    /// per the X027 two-state law), drives PetSim and answers every coin beat
    /// with the three-sync punch in the same frame. The theater placeholder row
    /// shows one slot per unlocked job; an assigned pet chip hops on the job -
    /// the "working" comedy face with zero dead ends (pack law).
    /// Slice B-1c: job slots are tappable (Button -&gt; TapJob -&gt; an idle pet
    /// reports for duty), the boot body lives in Initialize (idempotent) and
    /// the frame body in DriveFrame so the BotBarePlayCheck gate can drive
    /// the exact play path in batchmode; read-only probes expose the faces.
    /// &lt;/summary&gt;
    public sealed class PetMainHost : MonoBehaviour
    {
        const string CoinDingKey = "g15_sfx_coin";
        const float PulseDur = 0.35f;
        const float FlyDur = 0.9f;
        const int HookCoinRainBeats = 12;   // T+0-3s hook face: coin rain beat count (placeholder)
        PetSim _sim;
        Text _coinText;
        Text _sloganText;
        Text _flyText;
```

```
Transform _coinReadout;
Transform _theaterRow;
AudioSource _ding;
AudioClip _dingClip;
// same-frame three-sync state (applied in HandleCoinBeat, decayed in DriveFrame)
float _pulseAge = PulseDur;
float _flyAge = FlyDur;
double _coinShown;
// bot bare-play probe state (faces read via the probe properties below)
bool _booted;
int _coinBeatCount;           // beats answered by the view
string _lastFlyNumber = "";   // latest three-sync number face
string _lastTapRefusal = "";  // latest all-paws-busy comedy line (never a punishment)
// T003 slice B recruit + pet-dex faces (bot gate reads these via the probes below)
Text _recruitLabel;               // recruit button face (next pet + cost)
Text _dexCountFace;                // dex button face (open count / total)
GameObject _dexPage;              // album page overlay, toggled by TapDex
readonly System.Collections.Generic.List&lt;Text&gt; _dexCellTexts =
    new System.Collections.Generic.List&lt;Text&gt;();
bool _dexOpen;
string _lastRecruitFly = "";      // latest recruit success face
string _lastRecruitRefusal = "";  // latest recruit comedy refusal (zero punishment)
int _lastRecruitPetId = -1;
// T004 slice B job-board faces (bot gate reads these via the probes below)
Text _jobsLabel;                  // jobs button face (open count / total)
GameObject _jobsPage;             // job board overlay, toggled by TapJobs
readonly System.Collections.Generic.List&lt;Text&gt; _jobCellTexts =
    new System.Collections.Generic.List&lt;Text&gt;();
bool _jobsOpen;
string _lastJobFly = "";          // latest job-open success face
string _lastJobRefusal = "";     // latest job-board comedy refusal (zero punishment)
int _lastOpenedJobId = -1;
// T004 slice C: shop promote faces (bot gate reads these via the probes below)
Text _promoteLabel;                // promote button face (next rank / remaining gates)
string _lastPromoteFly = "";       // latest promote success face
string _lastPromoteRefusal = "";   // latest promote comedy refusal (zero punishment)
// T011: promotion stamp ceremony (SYS05) - the five-stage shop re-skin face
// and the ceremony overlay with its timed show beats. Floating law: the
// ceremony never closes another page (the event card keeps top draw).
Text _shopStageFace;               // HUD shop stage name (five-rank re-skin, M04)
GameObject _ceremonyPage;          // stamp ceremony overlay, opened by HandlePromoted
Text _ceremonyTitleFace;           // rank-up title (promote_toast routed)
Text _ceremonyStageFace;           // old -&gt; new stage face (five-stage re-skin)
Text _ceremonyStampFace;           // stamp beat face (pending until the stamp slams)
Text _ceremonyUnlockFace;          // unlock beat face (new posts = ticket to new play)
Text _ceremonyCrewFace;            // crew beat face (pets drawn to the new posts)
Text _ceremonyCollectLabel;        // collect button label (ceremony_collect routed)
bool _ceremonyOpen;
bool _ceremonyStampShown;           // stamp beat fired (     1)
bool _ceremonyUnlockShown;          // unlock beat fired (     2)
```

```
float _ceremonyT;                    // ceremony-local clock (DriveFrame dt accumulation)
string _lastCeremonyCollectFly = "";  // latest collect fly face (pure applause, zero writes)
int _promoteFanfareCount;            // AU07 placeholder beats (one per ceremony open)
const float CeremonyStampAtSec = 0.6f;    // stamp slam beat (    )
const float CeremonyUnlockAtSec = 1.2f;   // unlock + crew beat (    )
const string PromoteFanfareKey = "g15_sfx_promote";   // AU07 stem (audio batch swaps this placeholder)
AudioClip _promoteFanfareClip;
// T013: endgame finale overlay (M10) - full rank + every post lit opens the
// grand celebration once per save file. Floating law like the ceremony (it
// never closes another page); show beats are frame-driven clocks; the
// collect tap is pure applause. The annual report share (slice B) rides
// this page later.
GameObject _endgamePage;             // finale overlay, opened by HandleEndgameReady
Text _endgameTitleFace;              // finale title (endgame_title routed)
Text _endgameAllLitFace;             // all-lit stamp beat face (beat 1)
Text _endgameCrewFace;               // full-crew parade beat face (beat 2)
Text _endgameCollectLabel;           // collect button label (endgame_collect routed)
bool _endgameOpen;
bool _endgameAllLitShown;            // all-lit beat fired (     1)
bool _endgameCrewShown;              // crew parade beat fired (     2)
float _endgameT;                     // finale-local clock (DriveFrame dt accumulation)
string _lastEndgameCollectFly = ""; // latest collect fly face (pure applause, zero writes)
int _endgameFanfareCount;             // AU07 placeholder beats (one per finale open)
const float EndgameAllLitAtSec = 0.6f;   // all-lit stamp beat (    )
const float EndgameCrewAtSec = 1.2f;      // crew parade beat (    )
const string EndgameFanfareKey = "g15_sfx_endgame";   // AU stem (audio batch swaps this placeholder)
// T013 slice B: annual report long image (M10 share hammer) - the finale
// overlay carries a share button; the RT compose rides the same Sy.Social
// rig as the postcard card (zero server, red line 12).
Text _endgameShareLabel;                // finale share button label (annual_share routed)
int _annualShareCount;
bool _lastAnnualShareOk;
string _lastAnnualShareFace = "";
string _lastAnnualSharePath = "";
string _lastAnnualShareTitle = "";
string _lastAnnualShareLines = "";
string _lastAnnualShareQuery = "";
int _lastAnnualShareBytes;
int _lastAnnualShareWidth;
int _lastAnnualShareHeight;
float _lastAnnualShareMs;
// T013 slice D: NG+ view entry - the finale overlay carries the new-run
// button (the forward path: collections carry per pack section 11, the
// economic ladder resets to a fresh boot). Closed taps stay silent.
Text _endgameNgPlusLabel;              // finale new-run button label (ngplus_run routed)
string _lastNgPlusFly = "";            // latest restart fly face
int _ngPlusTapCount;                   // effective new-run taps (closed taps never count)
PetSim.NgPlusResult? _lastNgPlusResult; // latest new-run verdict (null = never tapped)
// U204 slice 6: perk pick sub-face on the finale overlay - the restart
// seat becomes a three-lane cultivation choice (crew / album / rep,
```

```
// pack section 11). PerkPickLive ships true (b-2 flip, X1020 pattern):
// the pick beat is the shipped face; the checks drive the crew lane
// answer, so the E32-E38 laws hold unchanged.
public static bool PerkPickLive = true;
GameObject _perkPickRow;              // pick sub-panel (child of the finale overlay)
Text _perkPickTitleFace;
Text _perkPickCrewFace;
Text _perkPickAlbumFace;
Text _perkPickRepFace;
int _perkPickOpenCount;
PetData.NgPlusPerk _lastNgPlusPickedPerk = (PetData.NgPlusPerk)(-1);   // latest lane tap (-1 = never)
// T005 slice B: offline return-visit page faces (bot gate reads these via probes)
GameObject _offlinePage;            // return-visit modal, fed by ShowOfflineWelcome
Text _offlineWelcomeFace;           // composer face 1: welcome line
Text _offlineGainFace;              // composer face 2: the credited wage receipt
Text _offlineDoubleFace;             // composer face 3: x2 offer button (claim / claimed)
bool _offlineOpen;
string _lastOfflineDoubleFly = ""; // latest x2 claim fly face
double _lastSettleCoin = -1.0;      // latest receipt coin (-1 = no page composed yet)
// T026 slice B: daily sign-in View faces (SYS retention beat; the popup
// presents right after the boot offline receipt closes - the return-visit
// beat hands off to the sign-in beat - and the home badge keeps the daily
// hook reachable). The sim owns every verdict (PeekSignIn / TryClaimSignIn);
// the View never re-derives the day or streak laws.
GameObject _signInPage;             // sign-in popup modal (one modal at a time)
GameObject _signInBadgeGo;          // home badge seat (hidden while claimed today)
GameObject _signInX2Go;             // T026-c2: popup done-face x2 seat (hidden while claimable)
Text _signInBadgeFace;
Text _signInTitleFace;
Text _signInDayFace;
Text _signInRewardFace;
Text _signInClaimFace;
Text _signInX2Face;                 // T026-c2: x2 seat face (call / done)
bool _signInOpen;
bool _signInX2OfferShown;           // T026-c2: evt_ad offer dedupe - one write per visible callable epoch
string _lastSignInFly = "";        // latest claim fly face
long _wallProbeMs = -1L;            // gate wall-feed stub (&gt;= 0 overrides the real clock; -1 = live)
// T006 slice B: comedy event card pop faces (bot gate reads these via probes)
GameObject _eventCardPage;          // event card overlay, popped by the resolve beat
Text _eventCardTitleFace;           // composer face 1: the comedy line
Text _eventCardGainFace;            // composer face 2: the stardust receipt
bool _eventCardOpen;
string _lastEventFly = "";          // latest collect fly face
int _lastEventCardId = -1;
int _eventPopCount;                 // pops answered by the view (per resolve)
double _lastEventStarDelta;         // latest card star-dust value (collect fly feed)
// T007 slice B: IAA triple offer faces - the rescue HUD seat, per-job speed
// seats on occupied theater slots, and the offline x2 re-presentation chip
// (pack section 7). The bot gate reads every face through the probes below.
const string RescuePointId = "g15_rescue";
```

```
const string SpeedPointId = "g15_speed";
const string DoublePointId = "g15_double";
const string SignInPointId = "g15_signin";   // T026-c2: sign-in x2 seat (AdCategories.G15SignIn, cap 1/day)
const string BoostPopKey = "g15_sfx_event_pop";          // AU06 stem (audio batch swaps this placeholder)
const string DoubleChimeKey = "g15_sfx_offline_double";  // AU09 stem (audio batch swaps this placeholder)
const float DoubleRePresentDelaySec = 1.0f;   // pack section 7: the chip presents inside the 1-5s anchor
const float DoublePresentWindowSec = 200f;    // pack section 7: loud presentation window, quiet badge after
Sy.Platform.AdCapPolicy _adCap;
Sy.Telemetry.TelemetryService _telemetry;
Sy.Core.ILocaleService _locale;   // T009 slice B: copy-table locale service (boot-registered, local fallback)
Text _rescueLabel;                    // rescue seat face: call / countdown / honest cap state
GameObject _doubleChipGo;             // offline x2 re-presentation chip (tap reopens the receipt page)
Text _doubleChipFace;
AudioClip _boostPopClip;
AudioClip _doubleChimeClip;
readonly System.Collections.Generic.List&lt;Text&gt; _speedCellTexts =
    new System.Collections.Generic.List&lt;Text&gt;();
readonly System.Collections.Generic.List&lt;string&gt; _funnel =
    new System.Collections.Generic.List&lt;string&gt;();   // evt_ad state writes (offer/show/complete/landed ch
bool _rescueOfferShown;               // evt_ad offer dedupe: one write per visible callable epoch
bool _speedOfferShown;
bool _doubleOfferShown;
bool _boostEpochSeen;                 // any-window epoch: rebuild trigger for the theater speed seats
int _lastFaceSec = -1;
double _chipArmedAt = -1.0;           // sim-time the chip window armed (close with a pending slot)
string _lastBoostFly = "";            // latest rescue/speed grant fly face
string _lastBoostRefusal = "";        // latest offer comedy refusal (zero punishment)
int _boostPopCount;                   // AU06 placeholder beats (rescue/speed grants)
int _doubleChimeCount;                // AU09 placeholder beats (double claims)
// T008 slice B-1: wage postcard album faces (bot gate reads these via probes).
// Channel A (the RT-composed card image + F4 pixel canary) lands slice B-2;
// this slice is the album page + channel B share-code faces + the away
// summary postcard strip on the offline page (M08 "crew writes home").
GameObject _postcardPage;            // postcard album overlay, toggled by TapCards
Text _cardsCountFace;                 // HUD button face (journal count / capacity)
Text _postcardCellHint;               // album honest-zero / window hint face
Text _postcardCodeFace;               // channel B: offline share code face
Text _postcardImportFace;             // channel B: locally re-derived import face
Text _offlineMailFace;                // away summary postcard strip on the offline page
bool _cardsOpen;
string _lastPostcardFly = "";        // latest grant / mail fly face
int _postcardPopCount;                 // journal sends answered by the view
readonly System.Collections.Generic.List&lt;Text&gt; _postcardCellTexts =
    new System.Collections.Generic.List&lt;Text&gt;();
// T008 slice B-2: channel A share-image state - the latest journal card
// composes into a local PNG through the shared Sy.Social RT rig (zero
// server, red line 12); the journal record stays the only data feed.
readonly Sy.Social.ShareCardService _shareCardService = new Sy.Social.ShareCardService();
bool _lastShareImageOk;
string _lastShareImageFace = "";
```

```
string _lastShareImagePath = "";
string _lastShareImageTitle = "";
string _lastShareImageLines = "";
string _lastShareImageQuery = "";
int _lastShareImageBytes;
int _lastShareImageWidth;
int _lastShareImageHeight;
float _lastShareImageMs;
int _shareImageCount;
// T012: honor wall faces (SYS08) - the achievement unlock beat renders the
// fly face + wall refresh; evt_zero_ad_clear telemetry rides the U091 honor.
const string ZeroAdClearEventId = "evt_zero_ad_clear";   // requirement doc section 8 telemetry
GameObject _honorPage;                  // honor wall overlay, toggled by TapHonors
Text _honorsLabel;                      // honors button face (unlocked count / total)
readonly System.Collections.Generic.List&lt;Text&gt; _honorCellTexts =
    new System.Collections.Generic.List&lt;Text&gt;();
bool _honorOpen;
string _lastAchFly = "";                // latest honor unlock fly face
int _achPopCount;                       // unlocks answered by the view
int _lastAchId = -1;                    // latest unlock ordinal (probe face)
int _zeroAdClearWrites;                 // evt_zero_ad_clear telemetry writes
int _signInWrites;                      // T026 evt_signin telemetry writes (probe face)
int _adSignInWrites;                    // T026-c evt_ad_signin telemetry writes (probe face)
readonly System.Collections.Generic.List&lt;Transform&gt; _petChips =
    new System.Collections.Generic.List&lt;Transform&gt;();
/// &lt;summary&gt;T+0-3s hook slogan (one-line pitch, requirement doc section 1).
/// T009 slice B: canonical copy faces read the copy table through the live
/// language (zero literal UI text; CopyTableCheck scans the law).&lt;/summary&gt;
public static string ComposeHookSlogan() =&gt; CopyTable.Get(CopyLocaleService.CurrentLang, "slogan");
/// &lt;summary&gt;Placeholder ding key + pulse scale + flying number, one call = three faces.&lt;/summary&gt;
public static CoinFeedback ComposeCoinFeedback(float t, double delta, int jobId)
{
    return new CoinFeedback(
        CoinDingKey,
        1.35f,
        "+" + delta.ToString("0.##", CultureInfo.InvariantCulture),
        jobId,
        t);
}
/// &lt;summary&gt;
/// Theater opening face for a dispatch (pet reports for duty). Labels derive
/// from the frozen slice-A schema ids ("job_coffee" -&gt; "coffee") so the data
/// mirror stays untouched; the art batch swaps in display-name faces later.
/// &lt;/summary&gt;
public static TheaterOpen ComposeTheaterOpen(int jobId, int petId)
{
    return new TheaterOpen(JobLabel(jobId), PetLabel(petId), jobId, petId);
}
static string JobLabel(int jobId)
{
```

```
    if (jobId &lt; 0 || jobId &gt;= PetData.JobCount) return "job";
    string id = PetData.Jobs[jobId].Id;
    return id != null &amp;&amp; id.StartsWith("job_") ? id.Substring(4) : id;
}
static string PetLabel(int petId)
{
    if (petId &lt; 0 || petId &gt;= PetData.PetCount) return "idle";
    string id = PetData.PetIds[petId];
    return id != null &amp;&amp; id.StartsWith("pet_") ? id.Substring(4) : id;
}
/// &lt;summary&gt;
/// T004 slice C shop-ladder promote button face (SYS05: five-rank door, never a
/// wall). Ready = call to action; otherwise the face shows only what is still
/// missing, so the ladder reads as progress instead of a lock. Gate order
/// mirrors the sim (jobs / pals / stars) - the bot gate asserts these strings.
/// &lt;/summary&gt;
public static string ComposePromoteFace(int rankIndex, int jobsOpen, int dexCount, int starLevel)
{
    if (rankIndex &gt;= PetData.RankCount - 1) return "Top rank R" + PetData.RankCount + "!";
    int g = rankIndex;   // gate from rank g to g+1
    int posts = PetData.GateJobsAtLeast[g] - jobsOpen;
    int pals = PetData.GateDexAtLeast[g] - dexCount;
    int stars = PetData.GateStarsAtLeast[g] - starLevel;
    if (posts &lt;= 0 &amp;&amp; pals &lt;= 0 &amp;&amp; stars &lt;= 0) return "Promote to R" + (rankIndex + 2)
    var sb = new System.Text.StringBuilder();
    sb.Append("R").Append(rankIndex + 2).Append(":");
    if (posts &gt; 0) sb.Append(" +").Append(posts).Append(posts &gt; 1 ? " posts" : " post");
    if (pals &gt; 0) sb.Append(" +").Append(pals).Append(pals &gt; 1 ? " pals" : " pal");
    if (stars &gt; 0) sb.Append(" +").Append(stars).Append(stars &gt; 1 ? " stars" : " star");
    return sb.ToString();
}
/// &lt;summary&gt;Promote refusal face: only the missing gate segments, sim gate order.&lt;/summary&gt;
public static string ComposePromoteRefusal(int rankIndex, int jobsOpen, int dexCount, int starLevel)
{
    if (rankIndex &gt;= PetData.RankCount - 1) return "Top rank already!";
    var sb = new System.Text.StringBuilder("Need ");
    bool any = false;
    int posts = PetData.GateJobsAtLeast[rankIndex] - jobsOpen;
    int pals = PetData.GateDexAtLeast[rankIndex] - dexCount;
    int stars = PetData.GateStarsAtLeast[rankIndex] - starLevel;
    if (posts &gt; 0) { sb.Append(posts).Append(posts &gt; 1 ? " posts" : " post"); any = true; }
    if (pals &gt; 0) { if (any) sb.Append(", "); sb.Append(pals).Append(pals &gt; 1 ? " pals" : " pal"); any = t
    if (stars &gt; 0) { if (any) sb.Append(", "); sb.Append(stars).Append(stars &gt; 1 ? " stars" : " star"); an
    return any ? sb.Append("!").ToString() : "Need nothing!";
}
/// &lt;summary&gt;Promote success fly face for the new rank index (rank 1 -&gt; "Rank up! R2!").
/// T009 slice B: the toast template lives in the copy table (promote_toast).&lt;/summary&gt;
public static string ComposePromoteFly(int newRankIndex)
{
    return CopyTable.Get(CopyLocaleService.CurrentLang, "promote_toast",
```

```
        (newRankIndex + 1).ToString());
}
/// &lt;summary&gt;T011 shop stage name (M04 five-stage re-skin): the HUD face that
/// renames the shop per rank. Routed from the copy table (shop_stage_rN).&lt;/summary&gt;
public static string ComposeShopStageFace(int rankIndex)
{
    if (rankIndex &lt; 0) rankIndex = 0;
    if (rankIndex &gt;= PetData.RankCount) rankIndex = PetData.RankCount - 1;
    return CopyTable.Get(CopyLocaleService.CurrentLang,
        "shop_stage_r" + (rankIndex + 1).ToString());
}
/// &lt;summary&gt;T011 ceremony stage face: the old -&gt; new stage rename line.&lt;/summary&gt;
public static string ComposeCeremonyStageFace(int oldRankIndex, int newRankIndex)
{
    return ComposeShopStageFace(oldRankIndex) + " -&gt; " + ComposeShopStageFace(newRankIndex);
}
/// &lt;summary&gt;T011 stamp beat face (ceremony_stamp): the promotion seal slam.&lt;/summary&gt;
public static string ComposeCeremonyStampFace(int newRankIndex)
{
    return CopyTable.Get(CopyLocaleService.CurrentLang, "ceremony_stamp",
        (newRankIndex + 1).ToString());
}
/// &lt;summary&gt;T011 unlock beat face (ceremony_unlock): promotion = a ticket
/// to new play - the posts that became openable at the new rank.&lt;/summary&gt;
public static string ComposeCeremonyUnlockFace(int newRankIndex)
{
    var parts = new System.Collections.Generic.List&lt;string&gt;();
    for (int j = 0; j &lt; PetData.JobCount; j++)
        if (PetData.Jobs[j].UnlockRank == newRankIndex) parts.Add(JobLabel(j));
    return CopyTable.Get(CopyLocaleService.CurrentLang, "ceremony_unlock",
        string.Join(", ", parts.ToArray()));
}
/// &lt;summary&gt;T011 crew beat face (ceremony_crew): the pets whose affinity
/// post just opened at the new rank (honest data - the pool grows with play).&lt;/summary&gt;
public static string ComposeCeremonyCrewFace(int newRankIndex)
{
    var parts = new System.Collections.Generic.List&lt;string&gt;();
    for (int p = 0; p &lt; PetData.PetCount; p++)
        if (PetData.PetDex[p].AffinityJob &gt;= 0
            &amp;&amp; PetData.Jobs[PetData.PetDex[p].AffinityJob].UnlockRank == newRankIndex)
            parts.Add(PetLabel(p));
    return CopyTable.Get(CopyLocaleService.CurrentLang, "ceremony_crew",
        string.Join(", ", parts.ToArray()));
}
/// &lt;summary&gt;T011 collect fly face (ceremony_collect): pure applause, zero writes.&lt;/summary&gt;
public static string ComposeCeremonyCollectFly()
{
    return CopyTable.Get(CopyLocaleService.CurrentLang, "ceremony_collect");
}
/// &lt;summary&gt;Return-visit welcome line (T005 slice B composer face 1).
```

```
/// T009 slice B: the line reads the copy table (offline_summary).&lt;/summary&gt;
public static string ComposeOfflineWelcomeFace()
{
    return CopyTable.Get(CopyLocaleService.CurrentLang, "offline_summary");
}
/// &lt;summary&gt;Return-visit receipt face (T005 slice B composer face 2): "+X c +Y stardust".&lt;/summary&gt;
public static string ComposeOfflineGainFace(double coin, double stardust)
{
    return "+" + coin.ToString("0.##", CultureInfo.InvariantCulture)
         + " c +" + stardust.ToString("0.##", CultureInfo.InvariantCulture) + " stardust";
}
/// &lt;summary&gt;Return-visit x2 offer button face (T005 slice B composer face 3): claim / claimed.&lt;/summar
public static string ComposeOfflineDoubleFace(bool claimable)
{
    return claimable ? "x2 wages!" : "Claimed!";
}
/// &lt;summary&gt;x2 claim fly face: the bonus amount rides the x2 hook.&lt;/summary&gt;
public static string ComposeOfflineDoubleFly(double bonus)
{
    return "+" + bonus.ToString("0.##", CultureInfo.InvariantCulture) + "c x2!";
}
/// &lt;summary&gt;
/// Event card comedy title face (T006 slice B composer face 1). Lines map to
/// the frozen pool ids through the copy table (T009 slice B) - the E04-E10
/// batch lands as pure table rows with zero view changes.
/// &lt;/summary&gt;
public static string ComposeEventCardTitle(int eventId)
{
    string id = eventId &gt;= 0 &amp;&amp; eventId &lt; PetData.EventCount ? PetData.Events[eventId].Id : "";
    return CopyTable.HasKey(id)
        ? CopyTable.Get(CopyLocaleService.CurrentLang, id)
        : "Something funny happened!";
}
/// &lt;summary&gt;Event card gain face (T006 slice B composer face 2): "+8 stardust".&lt;/summary&gt;
public static string ComposeEventGainFace(double starDelta)
{
    return "+" + starDelta.ToString("0.##", CultureInfo.InvariantCulture) + " stardust";
}
/// &lt;summary&gt;
/// Collect fly face: the stardust settled on the resolve frame, so the tap is
/// the applause - the face only reinforces the beat, never re-pays it.
/// &lt;/summary&gt;
public static string ComposeEventCollectFly(double starDelta)
{
    return ComposeEventGainFace(starDelta) + "!";
}
/// &lt;summary&gt;
/// U204 choice card title face: routes the choice pool ids through the copy
/// table (slice B ships the two-button card; the arms ride ComposeChoiceArmFace).
/// &lt;/summary&gt;
```

```
public static string ComposeChoiceCardTitle(int choiceId)
{
    string id = choiceId &gt;= 0 &amp;&amp; choiceId &lt; PetData.ChoiceCount ? PetData.ChoiceEvents[choiceId].I
    return CopyTable.HasKey(id)
        ? CopyTable.Get(CopyLocaleService.CurrentLang, id)
        : "The crew needs a call!";
}
/// &lt;summary&gt;
/// U204 arm face: the flavor routes the copy table, the stardust number
/// mirrors the def (single source of truth), and the B arm spells out the
/// album keepsake so the tap is weighted with consequences (R161 meaning law).
/// &lt;/summary&gt;
public static string ComposeChoiceArmFace(int choiceId, bool armA)
{
    if (choiceId &lt; 0 || choiceId &gt;= PetData.ChoiceCount) return "";
    var d = PetData.ChoiceEvents[choiceId];
    string flavor = CopyTable.Get(CopyLocaleService.CurrentLang, d.Id + (armA ? "_a" : "_b"));
    double star = armA ? d.ChoiceAStar : d.ChoiceBStar;
    string tail = !armA &amp;&amp; d.ChoiceBCollect ? " + album" : "";
    return flavor + "\n+" + star.ToString("0.##", CultureInfo.InvariantCulture) + " stardust" + tail;
}
/// &lt;summary&gt;
/// Rescue HUD seat face (T007 slice B): call-to-action while the category
/// has budget, live-window countdown while boosted, honest cap state after
/// the daily 5-view cap (pack section 7; the seat never hides a verdict).
/// &lt;/summary&gt;
public static string ComposeRescueOfferFace(bool windowActive, double remainSec, bool budgetLeft)
{
    if (windowActive)
        return "x2 on! " + (int)System.Math.Ceiling(remainSec / 60.0) + "m";
    return budgetLeft ? "Rescue! x2 30m" : "Cap hit! Tomorrow!";
}
/// &lt;summary&gt;Speed seat face on an occupied theater slot: live window / callable offer.&lt;/summary&gt;
public static string ComposeSpeedCellFace(bool boosted)
{
    return boosted ? "x2 on!" : "Speed x2";
}
/// &lt;summary&gt;
/// Offline x2 re-presentation chip face (pack section 7 "1-5/200" law):
/// blank in the first second after the close, the loud "x2 wages!" call
/// inside the 200s presentation window, then a quiet "x2?" badge that
/// stays tappable until the slot is claimed - the offer never silently
/// vanishes (U091: an optional bonus is reachable, never a hostage).
/// &lt;/summary&gt;
public static string ComposeDoubleChipFace(double elapsedSec)
{
    if (elapsedSec &lt; DoubleRePresentDelaySec) return "";
    return elapsedSec &lt; DoubleRePresentDelaySec + DoublePresentWindowSec ? "x2 wages!" : "x2?";
}
/// &lt;summary&gt;R160 idle-reassign revisit badge: count-only prompt, empty at full
```

```
}
void RebuildTheaterRow()
{
    for (int i = _petChips.Count - 1; i &gt;= 0; i--) DestroyFace(_petChips[i].gameObject);
    _petChips.Clear();
    for (int i = _speedCellTexts.Count - 1; i &gt;= 0; i--)
        if (_speedCellTexts[i] != null) DestroyFace(_speedCellTexts[i].gameObject);
    _speedCellTexts.Clear();
    for (int j = 0; j &lt; PetData.JobCount; j++) _speedCellTexts.Add(null);
    bool anyCallableSeat = false;
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    bool speedBudget = _adCap != null
        &amp;&amp; _adCap.HasCategoryBudget(day, Sy.Platform.AdCategories.G15Speed);
    int slotIndex = 0;
    for (int j = 0; j &lt; PetData.JobCount; j++)
    {
        if (!_sim.IsJobUnlocked(j)) continue;
        var open = ComposeTheaterOpen(j, _sim.JobOccupant(j));
        var slot = MakePanel(_theaterRow, new Vector2(-260f + 200f * slotIndex, 0f), new Vector2(180f, 280f), ne
        var btn = slot.AddComponent&lt;Button&gt;();   // slice B-1c: the slot itself is the tap face
        int tapped = j;
        btn.onClick.AddListener(() =&gt; TapJob(tapped));
        slotIndex++;
        MakeText(slot.transform, open.JobLabel, 26, new Vector2(0f, 100f), Color.white);
        if (_sim.JobOccupant(j) &gt;= 0)
        {
            var chip = MakePanel(slot.transform, new Vector2(0f, -10f), new Vector2(96f, 96f), PetChipColor(_sim
            MakeText(chip.transform, open.PetLabel, 22, Vector2.zero, Color.black);
            _petChips.Add(chip.transform);
            // T007 slice B: speed offer seat on the occupied slot only - an
            // empty job can never waste an ad view (pre-offer gate by design).
            bool boosted = _sim.SpeedBoostActive(j);
            if (!boosted &amp;&amp; speedBudget &amp;&amp; _sim.SpeedBoostJob &lt; 0) anyCallableSeat = true;
            var speedGo = MakePanel(slot.transform, new Vector2(0f, -108f), new Vector2(120f, 44f),
                boosted ? new Color(0.2f, 0.62f, 0.28f, 0.95f) : new Color(0.85f, 0.45f, 0.12f, 0.95f));
            speedGo.AddComponent&lt;Button&gt;().onClick.AddListener(() =&gt; TapSpeedOffer(tapped));
            _speedCellTexts[j] = MakeText(speedGo.transform,
                ComposeSpeedCellFace(boosted), 15, Vector2.zero, Color.white);
        }
    }
    // evt_ad offer state for the speed family: one write per visible
    // callable epoch (any occupied seat with budget and no live window)
    if (anyCallableSeat &amp;&amp; !_speedOfferShown)
    {
        _speedOfferShown = true;
        Funnel("offer", SpeedPointId);
    }
    else if (!anyCallableSeat) _speedOfferShown = false;
}
static void DestroyFace(Object go)
```

```
{
    // edit-mode safe teardown (bot gate rebuilds the row in batchmode)
    if (Application.isPlaying) Destroy(go);
    else DestroyImmediate(go);
}
static Color PetChipColor(int petId)
{
    // placeholder pet chip tint by pet index (catalog sprite swap point)
    float h = (petId * 0.13f) % 1f;
    return Color.HSVToRGB(h, 0.55f, 0.95f);
}
void HandleCoinBeat(float t, double delta, int jobId)
{
    // three-sync: sound + pulse + flying number, all in this same frame
    var fb = ComposeCoinFeedback(t, delta, jobId);
    _coinBeatCount++;
    _lastFlyNumber = fb.NumberText;
    // audible face only at play time; the edit-mode bot gate skips it
    // (the ding contract itself is asserted by Redline11Check)
    if (_dingClip != null &amp;&amp; Application.isPlaying) _ding.PlayOneShot(_dingClip);
    _pulseAge = 0f;
    _flyAge = 0f;
    _flyText.text = fb.NumberText;
}
void HandleDispatched(int jobId, int petId, float t)
{
    // work theater opens on the dispatch frame (pet reports for duty)
    RebuildTheaterRow();
}
void Update()
{
    DriveFrame(Time.deltaTime);
}
/// &lt;summary&gt;One real view frame; the bot gate drives the same body with fixed dt.&lt;/summary&gt;
public void DriveFrame(float dt)
{
    _sim.Tick(dt);
    // T007 slice B: boost epoch flip (window open &lt;-&gt; closed) rebuilds the
    // theater speed seats + rescue face; per-second rescue countdown refresh
    bool epoch = _sim.RescueBoostActive || _sim.SpeedBoostJob &gt;= 0;
    if (epoch != _boostEpochSeen)
    {
        _boostEpochSeen = epoch;
        RebuildTheaterRow();
        RefreshBoostFaces();
    }
    int faceSec = (int)_sim.SimTime;
    if (faceSec != _lastFaceSec)
    {
        _lastFaceSec = faceSec;
```

```
    RefreshBoostFaces();
}
// T011 ceremony show beats: the stamp slams, then the unlock + crew
// faces pop - deterministic frame-driven clocks, one flip each
if (_ceremonyOpen)
{
    _ceremonyT += dt;
    int newRank = _sim != null ? _sim.RankIndex : 0;
    if (!_ceremonyStampShown &amp;amp;&amp;amp; _ceremonyT &gt;= CeremonyStampAtSec)
    {
        _ceremonyStampShown = true;
        _ceremonyStampFace.text = ComposeCeremonyStampFace(newRank);
    }
    if (!_ceremonyUnlockShown &amp;amp;&amp;amp; _ceremonyT &gt;= CeremonyUnlockAtSec)
    {
        _ceremonyUnlockShown = true;
        _ceremonyUnlockFace.text = ComposeCeremonyUnlockFace(newRank);
        _ceremonyCrewFace.text = ComposeCeremonyCrewFace(newRank);
    }
}
// T013 finale show beats: the all-lit stamp slams, then the crew parade
// face pops - deterministic frame-driven clocks, one flip each
if (_endgameOpen)
{
    _endgameT += dt;
    if (!_endgameAllLitShown &amp;amp;&amp;amp; _endgameT &gt;= EndgameAllLitAtSec)
    {
        _endgameAllLitShown = true;
        _endgameAllLitFace.text = ComposeEndgameAllLitFace(_sim.JobsOpenCount);
    }
    if (!_endgameCrewShown &amp;amp;&amp;amp; _endgameT &gt;= EndgameCrewAtSec)
    {
        _endgameCrewShown = true;
        var crew = new System.Collections.Generic.List&lt;string&gt;();
        for (int p = 0; p &lt; PetData.PetCount; p++)
            if (_sim.IsPetRecruited(p)) crew.Add(PetData.PetIds[p].Substring(4));
        _endgameCrewFace.text = ComposeEndgameCrewFace(string.Join(", ", crew.ToArray()));
    }
}
// x2 chip presentation window (pack section 7 "1-5/200" law): blank in
// the first second after the close, loud inside the 200s window, quiet
// badge after - tappable in every state until the slot is claimed
if (_sim.HasDoubleOfferPending &amp;amp;&amp;amp; _chipArmedAt &gt;= 0.0 &amp;amp;&amp;amp; !_offlineOpen)
{
    string chipFace = ComposeDoubleChipFace(_sim.SimTime - _chipArmedAt);
    if (_doubleChipFace.text != chipFace) _doubleChipFace.text = chipFace;
}
else if (_doubleChipFace.text.Length &gt; 0) _doubleChipFace.text = "";
// R160 revisit badge: idle count face - visible only while idle &gt; 0
// (opportunity prompt mirroring the OnIdlePetsChanged hook, never a penalty)
```

```
        string idleFace = ComposeIdleBadgeFace(_sim.IdlePetCount);
        if (_idleBadgeFace.text != idleFace) _idleBadgeFace.text = idleFace;
        // T026 slice B: sign-in badge + popup faces ride the same frame refresh
        // (the preview is pure - no writes; a midnight rollover flips the badge live)
        RefreshSignInFaces();
        // number face: the readout rolls toward the ledger value
        if (_coinShown &lt; _sim.Coin)
        {
            _coinShown += (_sim.Coin - _coinShown) * Mathf.Min(1f, 18f * dt);
            if (_sim.Coin - _coinShown &lt; 0.005) _coinShown = _sim.Coin;
            _coinText.text = _coinShown.ToString("0.##", CultureInfo.InvariantCulture);
        }
        // pulse face decay
        if (_pulseAge &lt; PulseDur)
        {
            _pulseAge += dt;
            float k = 1f - (_pulseAge / PulseDur);
            if (k &lt; 0f) k = 0f;
            _coinReadout.localScale = Vector3.one * (1f + 0.35f * k);
        }
        else _coinReadout.localScale = Vector3.one;
        // flying number face
        if (_flyAge &lt; FlyDur)
        {
            _flyAge += dt;
            float k = _flyAge / FlyDur;
            _flyText.rectTransform.anchoredPosition = new Vector2(0f, 130f + 90f * k);
            var c = _flyText.color;
            c.a = 1f - k;
            _flyText.color = c;
        }
        else if (_flyText.text.Length &gt; 0) _flyText.text = "";
        // theater placeholder: occupied pet chips hop (working comedy face)
        for (int i = 0; i &lt; _petChips.Count; i++)
        {
            if (_petChips[i] == null) continue;
            float hop = Mathf.Abs(Mathf.Sin((float)_sim.SimTime * 3.5f + i)) * 14f;
            _petChips[i].localPosition = new Vector3(0f, -10f + hop, 0f);
        }
    }
    // ----- procedural D-grade placeholder builders (U020; catalog swap later) -----
    static GameObject MakePanel(Transform parent, Vector2 pos, Vector2 size, Color color)
    {
        var go = new GameObject("panel", typeof(RectTransform), typeof(Image));
        go.transform.SetParent(parent, false);
        var rt = go.GetComponent&lt;RectTransform&gt;();
        rt.anchoredPosition = pos;
        rt.sizeDelta = size;
        var img = go.GetComponent&lt;Image&gt;();
        img.color = color;
```

```
            return go;
        }
        static Text MakeText(Transform parent, string content, int size, Vector2 pos, Color color)
        {
            var go = new GameObject("label", typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = go.GetComponent&lt;RectTransform&gt;();
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(800f, size + 12f);
            var txt = go.GetComponent&lt;Text&gt;();
            txt.text = content;
            txt.fontSize = size;
            txt.color = color;
            txt.alignment = TextAnchor.MiddleCenter;
            txt.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            return txt;
        }
    }
}
// PetSim.cs
using System;
using System.Globalization;
using System.Collections.Generic;
using System.Text;
namespace Sy.PetWorkCrew.Game
{
    /// &lt;summary&gt;
    /// T002 slice A: dispatch core loop simulation (pure C#, zero scene dependency,
    /// zero RNG - idle determinism). Law: state is a pure function of the (dt stream,
    /// op stream), so any two runs with identical inputs produce byte-identical
    /// fingerprints - the base the offline settle (T005) later proves isomorphic
    /// against. Red line 11 anchors owned by this slice: hook content lives from
    /// frame zero, the first dispatch answers with its first wage beat inside the
    /// 1.0s window with margin (pack section 9), every occupied job produces per
    /// the rate table with the log soft cap, empty jobs produce nothing and nothing
    /// ever punishes (flip-a-pet is a comedy beat, no dead end). Slice B (next
    /// round) wires the View (coin ding three-sync: sound + pulse + number, work
    /// theater placeholder) and the DispatchCheck / Redline11Check editor gates.
    /// &lt;/summary&gt;
    public sealed class PetSim
    {
        /// &lt;summary&gt;Wage settle cadence: one coin beat per second per occupied job.&lt;/summary&gt;
        public const float WageBeatPeriodSec = 1f;
        /// &lt;summary&gt;
        /// Beat carry seeded on assignment so the first wage beat lands at half the
        /// beat period: inside the 1.0s red-line window with margin even under frame
        /// jitter (pack section 9 anchor: first dispatch feedback &amp;lt;= 1.0s).
        /// &lt;/summary&gt;
        public const float FirstBeatCarrySeedSec = WageBeatPeriodSec * 0.5f;
        // ----- T005 pack section 6 mirrors (offline earnings law) -----
```

```
/// &lt;summary&gt;Offline production = online efficiency x 0.6 (pack: offline yield is 60% of online).&lt;/summ
public const double OfflineCoef = 0.6;
/// &lt;summary&gt;Offline wide cap = 12h (real-idle anchor; far looser than the genre's harsh 180min caps).&lt;
public const double OfflineCapSec = 12.0 * 3600.0;
/// &lt;summary&gt;Return-visit double offer multiplier, once per day (pack section 7; R73 red line: every ad ga
public const double OfflineDoubleMult = 2.0;
// ----- T006 pack section 5 mirrors (comedy event engine) -----
/// &lt;summary&gt;Scripted first event card beat: inside the &amp;lt;=10s red-line anchor with margin (pack sec
public const float FirstEventCardAtSec = 8f;
/// &lt;summary&gt;Online event cadence window low bound (pack section 5: one event every 180-300s).&lt;/summary
public const double EventCadenceMinSec = 180.0;
/// &lt;summary&gt;Online event cadence window high bound.&lt;/summary&gt;
public const double EventCadenceMaxSec = 300.0;
// ----- T007 pack section 7 mirrors (IAA triple: optional, x2-capped) -----
/// &lt;summary&gt;Ad gain red line: every ad boost multiplies wages by exactly x2
/// (pack section 7 law 1, R73 evidence "20x drops = ugly"). Boosts never
/// stack beyond it - rescue and speed windows cap at x2 together.&lt;/summary&gt;
public const double AdBoostAmp = 2.0;
/// &lt;summary&gt;Rescue / speed boost window: 30 minutes (pack section 7 table).&lt;/summary&gt;
public const double AdBoostDurSec = 1800.0;
// ----- live state -----
double _simTime;
double _coin;
double _starDust;
int _starLevel;
int _rankIndex;
int _dexCount;   // recruited pet count (determines the star dust dex bonus)
readonly bool[] _petRecruited = new bool[PetData.PetCount];
readonly bool[] _jobUnlocked = new bool[PetData.JobCount];
readonly int[] _jobLevel = new int[PetData.JobCount];
readonly int[] _jobPet = new int[PetData.JobCount];        // occupant pet id, -1 = empty
readonly double[] _jobBeatCarry = new double[PetData.JobCount];
bool _hasDoubleOfferPending;   // T005 return-visit double offer slot (one claim per settle)
// ----- T006 event engine state (pack section 5) -----
bool _firstEventFired;                    // scripted E01 boot beat fired
int _eventResolveCount;                   // resolved event ordinal (cadence roll seed)
double _nextEventAt = -1.0;               // sim-time of the next cadence beat; -1 = none scheduled
readonly bool[] _eventCollected = new bool[PetData.EventCount];   // event album bits (SYS08)
// ----- U204 choice-card state (R161 spectrum; data layer slice A) -----
readonly bool[] _choiceFlags = new bool[PetData.ChoiceFlagCount];      // story flags (B arms plant)
readonly bool[] _choiceCollected = new bool[PetData.ChoiceCount];     // choice album bits
int _choiceWindowLeft = -1;   // auto resolves left before the next choice roll; -1 = unrolled
int _choicePendingId = -1;     // open choice card; auto cadence pauses while &gt;= 0
int _choiceResolveCount;
// U204 FQ slice b (X1554 freeze): risk-arm wage-pause window per job -
// _simTime at which the job's wages resume; paused beats keep firing (Amount=0).
readonly double[] _jobRiskPauseUntil = new double[PetData.JobCount];
bool _lastChoiceArmA;
// ----- T007 IAA boost state (pack section 7; data face - the View owns offers) -----
double _rescueBoostUntil = -1.0;          // sim-time; shop-wide x2 wage window
```

```
int _speedBoostJob = -1;                  // -1 = none
double _speedBoostUntil = -1.0;          // single-job x2 wage window
// ----- T008 postcard state (pack section 8; data face - slice B renders cards) -----
readonly double[] _pairWorkSec = new double[PetData.PostcardPairCount];    // work-time progress bars (never dec
readonly bool[] _pairCardCollected = new bool[PetData.PostcardPairCount];  // album collect bits
readonly List&lt;PostcardRecord&gt; _postcards = new List&lt;PostcardRecord&gt;(16);   // bounded journal (capac
int _postcardsSent;   // journal ordinals (wage grants + away summary cards)
// ----- T012 achievement state (SYS08 honor wall; data face - the View renders) -----
readonly bool[] _achievementUnlocked = new bool[PetData.AchievementCount];
int _adsWatched;                 // rewarded views consumed on granted boosts/claims (zero-ad honor condition)
int _lastAchievementId = -1;     // latest unlock ordinal (-1 = none yet)
// ----- T013 endgame state (M10 finale; data face - the View renders the ceremony) -----
bool _endgameFired;              // one-shot ready event latch (fires once per run)
bool _endgameShown;              // the finale played (one ending per save file - M10 law)
// ----- T013 slice C: NG+ inheritance state (M10 second run; data face) -----
int _ngPlusCount;                // completed restarts that carried over (0 = first run)
// ----- U204 depth slice: NG+ inheritance cultivation perk (chosen per
// restart; shapes only the new run). Coefs stay 1.0 outside NG+ runs, so
// every normal-run face is an exact identity (byte-identical). -----
PetData.NgPlusPerk _ngPlusPerk = PetData.NgPlusPerk.None;
double _ngWageCoef = 1.0;
double _ngPairBarCoef = 1.0;
// ----- T026 sign-in retention state (pack section 12; R162 check-in
// spectrum; data face - slice B renders the boot popup). Wall-clock day
// domain fed by the caller (X995 law: uptime days are blind across
// sessions) - the sim never reads a clock itself. -1 = never claimed. -----
long _signInWallDay = -1;      // wall day index of the last claim
int _signInStreak;             // consecutive-day count (soft reset on a gap)
bool _signInClaimedToday;      // one claim per wall day
bool _signInDoubledToday;      // T026-c: the day's optional x2 double latch (resets on the next claim)
// ----- red line 11 timing capture -----
public float HookReadyT { get; private set; } = -1f;
public float FirstDispatchT { get; private set; } = -1f;
public float FirstWageBeatT { get; private set; } = -1f;
public float FirstRecruitT { get; private set; } = -1f;
public float FirstOpenJobT { get; private set; } = -1f;
public float FirstStarUpT { get; private set; } = -1f;
public float FirstPromoteT { get; private set; } = -1f;
public float FirstEventT { get; private set; } = -1f;   // T006: first event card (E01 first-laugh beat)
public float FirstAchievementT { get; private set; } = -1f;   // T012: first honor unlock (honor wall beat)
public int WageBeatCount { get; private set; }
/// &lt;summary&gt;First dispatch to first wage beat gap, -1 before the pair completes.&lt;/summary&gt;
public float DispatchToFirstBeatSec
    =&gt; (FirstDispatchT &gt;= 0f &amp;&amp; FirstWageBeatT &gt;= 0f) ? FirstWageBeatT - FirstDispatchT : -1f;
// ----- sim events (editor gates subscribe for timestamp-capture assertions) -----
public event Action&lt;float&gt; OnHookReady;                 // t
public event Action&lt;int, int, float&gt; OnDispatched;       // jobId, petId, t
public event Action&lt;int, float&gt; OnIdlePetsChanged;      // idleCount, t (R160 idle-reassign hook)
public event Action&lt;float, double, int&gt; OnCoinBeat;      // t, delta, jobId
public event Action&lt;int, float&gt; OnPetRecruited;         // petId, t
```

```
public event Action&lt;int, float&gt; OnPetGifted;            // petId, t (milestone grant channel, zero coin)
public event Action&lt;int, float&gt; OnJobOpened;            // jobId, t
public event Action&lt;int, int, float&gt; OnJobUpgraded;     // jobId, newLevel, t
public event Action&lt;int, float&gt; OnStarUp;               // starLevel, t
public event Action&lt;int, float&gt; OnPromoted;             // newRank, t
public event Action&lt;OfflineSettleResult, float&gt; OnOfflineSettled;   // settle record, t (T005)
public event Action&lt;double, float&gt; OnOfflineDoubleClaimed;          // bonus coin, t (T005)
public event Action&lt;int, string, double, float&gt; OnEventResolved;    // eventId, fx, starDelta, t (T006)
public event Action&lt;int, bool, double, float&gt; OnChoiceResolved;     // choiceId, armA, starDelta, t (U204;
public event Action&lt;int, float&gt; OnChoiceOpened;                     // choiceId, t (U204 slice B: the two-b
public event Action&lt;int, int, float&gt; OnAdBoostActivated;            // kind, jobId, t (T007; -1 = shop-wid
public event Action&lt;PostcardRecord, float&gt; OnPostcardSent;          // record, t (T008; wage grant or away
public event Action&lt;int, float&gt; OnAchievementUnlocked;              // achId, t (T012; honor wall beat)
public event Action&lt;float&gt; OnEndgameReady;                          // t (T013; full-rank all-lit finale, o
public event Action&lt;int, float&gt; OnNewGamePlusStarted;               // ngPlusCount, t (T013 slice C; the re
public event Action&lt;int, int, double, float&gt; OnSignInResolved;      // streak, cycleIndex, reward, t (T026
public event Action&lt;int, int, double, float&gt; OnSignInDoubled;       // streak, cycleIndex, bonus, t (T026-c
public PetSim()
{
    for (int i = 0; i &lt; PetData.JobCount; i++) _jobPet[i] = -1;
}
// ----- readouts -----
public double SimTime =&gt; _simTime;
public double Coin =&gt; _coin;
public double StarDust =&gt; _starDust;
public int StarLevel =&gt; _starLevel;
public int RankIndex =&gt; _rankIndex;
public int DexCount =&gt; _dexCount;
public bool IsPetRecruited(int petId) =&gt; _petRecruited[petId];
// ----- T003 pet album (SYS03/SYS08 data face): one dex page per pet, unlocks on recruit/gift -----
public bool PetAlbumUnlocked(int petId)
    =&gt; petId &gt;= 0 &amp;&amp; petId &lt; PetData.PetCount &amp;&amp; _petRecruited[petId];
public int PetAlbumOpenCount =&gt; _dexCount;
public bool PetAlbumFull =&gt; _dexCount == PetData.PetCount;
public bool IsJobUnlocked(int jobId) =&gt; _jobUnlocked[jobId];
public int JobLevel(int jobId) =&gt; _jobLevel[jobId];
public int JobOccupant(int jobId) =&gt; _jobPet[jobId];
public int JobsOpenCount { get; private set; }
// ----- T005 offline settle readouts (return-visit summary data face) -----
/// &lt;summary&gt;Last settle record; the View slice renders the return-visit receipt from it.&lt;/summary&gt;
public OfflineSettleResult LastOfflineSettle { get; private set; }
/// &lt;summary&gt;True when the last settle raised a claimable x2 offer slot.&lt;/summary&gt;
public bool HasDoubleOfferPending =&gt; _hasDoubleOfferPending;
// ----- T006 event engine readouts (data face; slice B renders the card pop layer) -----
/// &lt;summary&gt;Resolved event count (cadence ordinal).&lt;/summary&gt;
public int EventResolveCount =&gt; _eventResolveCount;
/// &lt;summary&gt;Event album (SYS08) collect bit per event id.&lt;/summary&gt;
public bool IsEventCollected(int eventId)
    =&gt; eventId &gt;= 0 &amp;&amp; eventId &lt; PetData.EventCount &amp;&amp; _eventCollected[eventId];
/// &lt;summary&gt;Collected event album entries.&lt;/summary&gt;
```

```
public int EventAlbumOpenCount
{
    get { int c = 0; for (int e = 0; e &lt; PetData.EventCount; e++) if (_eventCollected[e]) c++; return c; }
}
/// &lt;summary&gt;True when the whole event pool is in the album (SYS08 full face).&lt;/summary&gt;
public bool EventAlbumFull =&gt; EventAlbumOpenCount == PetData.EventCount;
// ----- U204 choice-card readouts (data face; slice B renders the two-button card) -----
/// &lt;summary&gt;Settled choice count (choice ordinal).&lt;/summary&gt;
public int ChoiceResolveCount =&gt; _choiceResolveCount;
/// &lt;summary&gt;True while a choice card pends (auto cadence pauses - one decision at a time).&lt;/summary&gt
public bool ChoicePending =&gt; _choicePendingId &gt;= 0;
/// &lt;summary&gt;Open choice id, -1 = none.&lt;/summary&gt;
public int ChoicePendingId =&gt; _choicePendingId;
/// &lt;summary&gt;Auto resolves left before the next choice roll; -1 = window not rolled yet.&lt;/summary&gt;
public int ChoiceWindowLeft =&gt; _choiceWindowLeft;
/// &lt;summary&gt;Arm taken on the latest settle (true = A).&lt;/summary&gt;
public bool LastChoiceArmA =&gt; _lastChoiceArmA;
/// &lt;summary&gt;Choice album bit per choice id (R161: the B arm feeds the album).&lt;/summary&gt;
public bool IsChoiceCollected(int choiceId)
    =&gt; choiceId &gt;= 0 &amp;&amp; choiceId &lt; PetData.ChoiceCount &amp;&amp; _choiceCollected[choiceId];
/// &lt;summary&gt;Story flag state (R161 callback arc: B arm plants, ReqFlag consumes).&lt;/summary&gt;
public bool IsChoiceFlagSet(int flagId)
    =&gt; flagId &gt;= 0 &amp;&amp; flagId &lt; PetData.ChoiceFlagCount &amp;&amp; _choiceFlags[flagId];
/// &lt;summary&gt;U204 FQ slice b (X1554 freeze): true while the job's wages sit
/// in a risk-arm pause window (the beat keeps firing, Amount=0 honest zero face).&lt;/summary&gt;
public bool RiskPauseActive(int job)
    =&gt; job &gt;= 0 &amp;&amp; job &lt; PetData.JobCount &amp;&amp; _simTime &lt; _jobRiskPauseUntil[job];
// ----- T007 IAA boost readouts (data face; the View/slice B renders offers) -----
/// &lt;summary&gt;Rescue boost kind id for the OnAdBoostActivated event.&lt;/summary&gt;
public const int BoostKindRescue = 0;
/// &lt;summary&gt;Speed boost kind id for the OnAdBoostActivated event.&lt;/summary&gt;
public const int BoostKindSpeed = 1;
/// &lt;summary&gt;True while the shop-wide rescue x2 wage window is open.&lt;/summary&gt;
public bool RescueBoostActive =&gt; _simTime &lt; _rescueBoostUntil;
/// &lt;summary&gt;Rescue window remaining seconds (0 when closed).&lt;/summary&gt;
public double RescueBoostRemainSec =&gt; RescueBoostActive ? _rescueBoostUntil - _simTime : 0.0;
/// &lt;summary&gt;Speed-boosted job id, -1 when no window is open.&lt;/summary&gt;
public int SpeedBoostJob =&gt; _speedBoostUntil &gt; _simTime ? _speedBoostJob : -1;
/// &lt;summary&gt;True while the given job's speed x2 window is open.&lt;/summary&gt;
public bool SpeedBoostActive(int jobId) =&gt; _speedBoostUntil &gt; _simTime &amp;&amp; _speedBoostJob == jobId;
// ----- T008 postcard readouts (data face; slice B renders album + cards) -----
/// &lt;summary&gt;Total postcards sent (wage grants + away cards).&lt;/summary&gt;
public int PostcardsSentTotal =&gt; _postcardsSent;
/// &lt;summary&gt;Bounded journal size (pack capacity law).&lt;/summary&gt;
public int PostcardJournalCount =&gt; _postcards.Count;
/// &lt;summary&gt;Journal entry accessor for gates and the View feed.&lt;/summary&gt;
public PostcardRecord PostcardAt(int index) =&gt; _postcards[index];
/// &lt;summary&gt;Most recent journal entry, null when empty.&lt;/summary&gt;
public PostcardRecord LastPostcard =&gt; _postcards.Count &gt; 0 ? _postcards[_postcards.Count - 1] : null;
/// &lt;summary&gt;Work-time progress bar of one (pet, job) pair, in seconds (never decays).&lt;/summary&gt;
```

```
public double PairPostcardProgressSec(int petId, int jobId)
    =&gt; (petId &gt;= 0 &amp;&amp; petId &lt; PetData.PetCount &amp;&amp; jobId &gt;= 0 &amp;&amp; jobId &lt; P
       ? _pairWorkSec[petId * PetData.JobCount + jobId] : 0.0;
/// &lt;summary&gt;Album collect bit of one (pet, job) pair card.&lt;/summary&gt;
public bool IsPairPostcardCollected(int petId, int jobId)
    =&gt; (petId &gt;= 0 &amp;&amp; petId &lt; PetData.PetCount &amp;&amp; jobId &gt;= 0 &amp;&amp; jobId &lt; P
       &amp;&amp; _pairCardCollected[petId * PetData.JobCount + jobId];
/// &lt;summary&gt;Collected pair cards (album open pages).&lt;/summary&gt;
public int PairPostcardOpenCount
{
    get { int c = 0; for (int k = 0; k &lt; PetData.PostcardPairCount; k++) if (_pairCardCollected[k]) c++; retu
}
/// &lt;summary&gt;True when the whole pair album is collected (SYS08 full face).&lt;/summary&gt;
public bool PairPostcardAlbumFull =&gt; PairPostcardOpenCount == PetData.PostcardPairCount;
// ----- T012 achievement readouts (data face; the View renders the honor wall) -----
/// &lt;summary&gt;Honor unlock bit per achievement id.&lt;/summary&gt;
public bool IsAchievementUnlocked(int achId)
    =&gt; achId &gt;= 0 &amp;&amp; achId &lt; PetData.AchievementCount &amp;&amp; _achievementUnlocked[achId];
/// &lt;summary&gt;Unlocked honor count (honor wall progress face).&lt;/summary&gt;
public int AchievementUnlockedCount
{
    get { int c = 0; for (int a = 0; a &lt; PetData.AchievementCount; a++) if (_achievementUnlocked[a]) c++; ret
}
/// &lt;summary&gt;True when the whole honor wall is gold (legend face).&lt;/summary&gt;
public bool AllAchievementsUnlocked =&gt; AchievementUnlockedCount == PetData.AchievementCount;
/// &lt;summary&gt;Rewarded ad views consumed on granted boosts/claims (U091 zero-ad honor condition face).&lt;/
public int AdsWatchedCount =&gt; _adsWatched;
/// &lt;summary&gt;Latest unlock ordinal, -1 before the first honor.&lt;/summary&gt;
public int LastAchievementId =&gt; _lastAchievementId;
// ----- T013 endgame readouts (M10: full-rank + every post lit finale) -----
/// &lt;summary&gt;Finale condition: top rank reached AND all jobs open (all posts lit).&lt;/summary&gt;
public bool EndgameReady =&gt; _rankIndex &gt;= PetData.RankCount - 1 &amp;&amp; JobsOpenCount &gt;= PetData.Job
/// &lt;summary&gt;The finale ceremony already played (one ending per save file - M10 law).&lt;/summary&gt;
public bool EndgameShown =&gt; _endgameShown;
/// &lt;summary&gt;One-way latch the View flips when it opens the finale (mark on open:
/// a mid-ceremony quit never replays the ending). Idempotent by design.&lt;/summary&gt;
public void MarkEndgameShown()
{
    if (_endgameShown) return;
    _endgameShown = true;
}
// ----- T026 sign-in retention readouts (pack section 12; data face) -----
/// &lt;summary&gt;Consecutive-day sign-in streak (0 = never claimed; soft reset on a gap).&lt;/summary&gt;
public int SignInStreak =&gt; _signInStreak;
/// &lt;summary&gt;Wall day index of the last claim (-1 = never).&lt;/summary&gt;
public long SignInWallDay =&gt; _signInWallDay;
/// &lt;summary&gt;True while today's claim is already taken (one claim per wall day).&lt;/summary&gt;
public bool SignInClaimedToday =&gt; _signInClaimedToday;
public bool SignInDoubledToday =&gt; _signInDoubledToday;   // T026-c: the View arm hides once the day is double
/// &lt;summary&gt;Cycle slot the next claim would pay: (streak) % 7 for a fresh
```

```
/// consecutive day, index of the streak-1 face after the claim. Probe face
/// for the gate; the reward itself comes from PetData.SignInRewardForStreak.&lt;/summary&gt;
public int SignInNextCycleIndex =&gt; _signInStreak % PetData.SignInCycleDays;
// ----- T013 slice C: NG+ inheritance (M10 second run; data face - the
// View slice owns the entry button and the carried-over celebration) -----
/// &lt;summary&gt;Completed restarts that carried over (0 = first run).&lt;/summary&gt;
public int NewGamePlusCount =&gt; _ngPlusCount;
/// &lt;summary&gt;U204 depth slice: the cultivation perk chosen for the current
/// NG+ run (None on first run and after a default restart).&lt;/summary&gt;
public PetData.NgPlusPerk NewGamePlusPerk =&gt; _ngPlusPerk;
public enum NgPlusResult { Ok = 0, NotReady = 1, InvalidPerk = 2 }
/// &lt;summary&gt;
/// T013 slice C: start the second run (unperked - byte-identical to the
/// slice C/D behavior; the perk lane lands below and folds to None).
/// Eligibility = the finale condition (top rank AND every post lit - pack
/// section 11 freeze). Carried faces ride the section 11 coefficients
/// (both machine-set 1.0 = full carry): pet dex album, event album,
/// postcard pair album, the bounded journal, the honor wall and the ads
/// counter - a restart never wipes a collection page (R72 / R73
/// anchors). Everything economic resets to a fresh boot: coin, star
/// dust, star level, rank ladder, job posts, dispatch seats, boost
/// windows, the offline offer slot, pair work bars, the red-line
/// first-anchors (the new run replays its own hook / first dispatch /
/// E01 first-laugh beats) and the finale latches - a new run is a new
/// ending slot (M10 one-ending-per-file law). Every refusal is a
/// zero-state-change verdict. Deterministic: the restart is a pure
/// function of the pre-state, so twin runs stay byte-identical across it.
/// &lt;/summary&gt;
public NgPlusResult TryStartNewGamePlus() =&gt; TryStartNewGamePlus(PetData.NgPlusPerk.None);
/// &lt;summary&gt;
/// U204 depth slice: restart with ONE inheritance cultivation perk chosen
/// for the new run (pack section 11 perk table). The perk shapes ONLY the
/// new run - crew wisdom: wage beats x1.5; photo memories: pair postcard
/// bars fill x2 (online beats and away credits); star reputation: the
/// new run opens at rank 1. Carried collections and the reset law are
/// untouched; an unknown perk id refuses with zero state change (the
/// refusal family law). The ad x2 red line is intact: the crew coef is
/// a base-rate face, and every ad boost still multiplies by exactly 2.
/// &lt;/summary&gt;
public NgPlusResult TryStartNewGamePlus(PetData.NgPlusPerk perk)
{
    if (!EndgameReady) return NgPlusResult.NotReady;   // refused, zero state change
    bool knownPerk = false;
    for (int i = 0; i &lt; PetData.NgPlusPerks.Length; i++)
        if (PetData.NgPlusPerks[i].Perk == perk) { knownPerk = true; break; }
    if (!knownPerk) return NgPlusResult.InvalidPerk;    // refused, zero state change
    var perkDef = PetData.NgPlusPerkDefOf(perk);
    // economic reset (fresh boot faces)
    _simTime = 0.0;
    _coin = 0.0;
```

```
_starDust = 0.0;
_starLevel = 0;
_rankIndex = perkDef.StartRank;   // U204: rep perk opens at rank 1; None = fresh ladder
for (int j = 0; j &lt; PetData.JobCount; j++)
{
    _jobUnlocked[j] = false;
    _jobLevel[j] = 0;
    _jobPet[j] = -1;
    _jobBeatCarry[j] = 0.0;
}
JobsOpenCount = 0;
_hasDoubleOfferPending = false;
LastOfflineSettle = null;
_rescueBoostUntil = -1.0;
_speedBoostJob = -1;
_speedBoostUntil = -1.0;
_firstEventFired = false;
_eventResolveCount = 0;
_nextEventAt = -1.0;
LastEventCard = null;
// U204: choice machinery resets with the run (album bits carry like
// _eventCollected; story flags are run-state, so the arc replays)
_choiceWindowLeft = -1;
_choicePendingId = -1;
_choiceResolveCount = 0;
_lastChoiceArmA = false;
LastChoiceCard = null;
for (int f = 0; f &lt; PetData.ChoiceFlagCount; f++) _choiceFlags[f] = false;
// U204 FQ slice b: risk-arm pause windows are run-state (new run = clean wages).
for (int r = 0; r &lt; PetData.JobCount; r++) _jobRiskPauseUntil[r] = 0.0;
for (int k = 0; k &lt; PetData.PostcardPairCount; k++) _pairWorkSec[k] = 0.0;
_endgameFired = false;
_endgameShown = false;
HookReadyT = -1f;
FirstDispatchT = -1f;
FirstWageBeatT = -1f;
FirstRecruitT = -1f;
FirstOpenJobT = -1f;
FirstStarUpT = -1f;
FirstPromoteT = -1f;
FirstEventT = -1f;
FirstAchievementT = -1f;
WageBeatCount = 0;
// fresh-boot seed: the free coffee post re-opens (the starter cat is
// already on the dex from the carry - the recruit folds to a silent
// no-op by the carried bits, so the seed inlines the post open only)
_jobUnlocked[PetData.JobCoffee] = true;
_jobLevel[PetData.JobCoffee] = 1;
_jobPet[PetData.JobCoffee] = -1;
JobsOpenCount = 1;
```

```
    // U204: the chosen cultivation lane takes effect on the new run only
    _ngPlusPerk = perk;
    _ngWageCoef = perkDef.WageCoef;
    _ngPairBarCoef = perkDef.PairBarCoef;
    _ngPlusCount++;
    OnNewGamePlusStarted?.Invoke(_ngPlusCount, (float)_simTime);
    return NgPlusResult.Ok;
}
/// &lt;summary&gt;Event card data face the View pop layer (slice B) renders.&lt;/summary&gt;
public sealed class EventCard
{
    public int EventId;
    public string Id;        // localization key face (event_e01..)
    public string Fx;       // view fx stem
    public double StarDelta; // star dust settled by this occurrence
}
/// &lt;summary&gt;Last resolved card; the View slice shows one modal per resolve.&lt;/summary&gt;
public EventCard LastEventCard { get; private set; }
/// &lt;summary&gt;Latest settled choice card (U204); null before the first settle.&lt;/summary&gt;
public EventCard LastChoiceCard { get; private set; }
/// &lt;summary&gt;Occupied job count (derived, drift-proof across reassign/displace ops).&lt;/summary&gt;
public int OnDutyPetCount
{
    get
    {
        int c = 0;
        for (int j = 0; j &lt; PetData.JobCount; j++)
            if (_jobUnlocked[j] &amp;&amp; _jobPet[j] &gt;= 0) c++;
        return c;
    }
}
/// &lt;summary&gt;R160 idle-reassign hook: recruited pets not on any job. The
/// revisit badge / summary faces hang off this count + the change event to
/// prompt a re-dispatch decision (opportunity cost face, never a penalty).&lt;/summary&gt;
public int IdlePetCount
{
    get
    {
        int idle = 0;
        for (int p = 0; p &lt; PetData.PetCount; p++)
            if (_petRecruited[p] &amp;&amp; !IsPetOnDuty(p)) idle++;
        return idle;
    }
}
public bool IsPetOnDuty(int petId)
{
    for (int j = 0; j &lt; PetData.JobCount; j++)
        if (_jobPet[j] == petId) return true;
    return false;
}
```

```
// ----- op results (every refusal is a zero-state-change verdict) -----
public enum AssignResult { Ok = 0, PetLocked = 1, JobLocked = 2 }
public enum RecruitResult { Ok = 0, AlreadyRecruited = 1, NoCoin = 2 }
public enum OpenJobResult { Ok = 0, AlreadyOpen = 1, RankLocked = 2, NoCoin = 3 }
public enum UpgradeResult { Ok = 0, JobLocked = 1, NoCoin = 2, Capped = 3 }
public enum PromoteResult { Ok = 0, MaxRank = 1, GatesNotMet = 2 }
public enum BoostResult { Ok = 0, AdNotConsumed = 1, JobLocked = 2, JobEmpty = 3, AlreadyActive = 4 }
/// &lt;summary&gt;Boot seed: starter cat gift + free first job open (pack section 1/4).&lt;/summary&gt;
public void SeedStarterState()
{
    RecruitPet(PetData.PetOrange);
    OpenJob(PetData.JobCoffee);
}
public RecruitResult RecruitPet(int petId)
{
    if (petId &lt; 0 || petId &gt;= PetData.PetCount) return RecruitResult.NoCoin; // invalid id = refused, zero
    if (_petRecruited[petId]) return RecruitResult.AlreadyRecruited;
    double cost = PetData.RecruitCosts[petId];
    if (_coin &lt; cost) return RecruitResult.NoCoin;
    _coin -= cost;
    _petRecruited[petId] = true;
    _dexCount++;
    if (FirstRecruitT &lt; 0f) FirstRecruitT = (float)_simTime;
    OnPetRecruited?.Invoke(petId, (float)_simTime);
    OnIdlePetsChanged?.Invoke(IdlePetCount, (float)_simTime);   // R160: recruit adds an idle pet
    EvaluateAchievements();
    return RecruitResult.Ok;
}
/// &lt;summary&gt;
/// T003: milestone gift channel (pack section 4 - recruit = coin direct-buy +
/// milestone grants, zero gacha, zero probability). Grants a pet without
/// touching the coin balance; every refusal is a zero-state-change verdict.
/// Invalid ids fold into the NoCoin verdict (refusal family, zero change).
/// Fire-station companion wiring lands with the T004 job-pool slice which
/// owns OpenJob.
/// &lt;/summary&gt;
public RecruitResult GiftPet(int petId)
{
    if (petId &lt; 0 || petId &gt;= PetData.PetCount) return RecruitResult.NoCoin; // refused, zero change
    if (_petRecruited[petId]) return RecruitResult.AlreadyRecruited;
    _petRecruited[petId] = true;
    _dexCount++;
    if (FirstRecruitT &lt; 0f) FirstRecruitT = (float)_simTime;
    OnPetGifted?.Invoke(petId, (float)_simTime);
    OnIdlePetsChanged?.Invoke(IdlePetCount, (float)_simTime);   // R160: gift adds an idle pet
    EvaluateAchievements();
    return RecruitResult.Ok;
}
public OpenJobResult OpenJob(int jobId)
{
```

```
    if (jobId &lt; 0 || jobId &gt;= PetData.JobCount) return OpenJobResult.RankLocked; // invalid id = refused
    if (_jobUnlocked[jobId]) return OpenJobResult.AlreadyOpen;
    if (_rankIndex &lt; PetData.Jobs[jobId].UnlockRank) return OpenJobResult.RankLocked;
    double cost = PetData.Jobs[jobId].OpenCost;
    if (_coin &lt; cost) return OpenJobResult.NoCoin;
    _coin -= cost;
    _jobUnlocked[jobId] = true;
    _jobLevel[jobId] = 1;
    _jobPet[jobId] = -1;
    JobsOpenCount++;
    // Fire-station companion wiring (pack section 4: Awang is the fire-station
    // end-game companion pet): opening the terminal easter-egg job grants him
    // through the milestone gift channel - zero coin, and an already-recruited
    // Awang folds to a silent no-op so the egg never blocks or punishes.
    if (jobId == PetData.JobFire &amp;&amp; !_petRecruited[PetData.PetAwang])
        GiftPet(PetData.PetAwang);
    if (FirstOpenJobT &lt; 0f) FirstOpenJobT = (float)_simTime;
    OnJobOpened?.Invoke(jobId, (float)_simTime);
    EvaluateAchievements();
    return OpenJobResult.Ok;
}
/// &lt;summary&gt;
/// Dispatch op: move a recruited pet onto an unlocked job. Reassignment is free
/// of punishment (pack law: flipping a pet is a comedy beat, never a penalty):
/// the pet leaves its old job empty, the displaced occupant (if any) goes idle.
/// &lt;/summary&gt;
public AssignResult AssignPet(int jobId, int petId)
{
    if (jobId &lt; 0 || jobId &gt;= PetData.JobCount || petId &lt; 0 || petId &gt;= PetData.PetCount)
        return AssignResult.JobLocked; // invalid id = refused, zero change
    if (!_petRecruited[petId]) return AssignResult.PetLocked;
    if (!_jobUnlocked[jobId]) return AssignResult.JobLocked;
    for (int j = 0; j &lt; PetData.JobCount; j++)
    {
        if (_jobPet[j] == petId)
        {
            if (j == jobId) return AssignResult.Ok; // no-op re-dispatch
            _jobPet[j] = -1;
            _jobBeatCarry[j] = 0.0;
        }
    }
    // a displaced old occupant (if any) simply goes idle - comedy beat, never a penalty
    _jobPet[jobId] = petId;
    _jobBeatCarry[jobId] = FirstBeatCarrySeedSec;   // first beat inside the 1.0s window
    if (FirstDispatchT &lt; 0f) FirstDispatchT = (float)_simTime;
    OnDispatched?.Invoke(jobId, petId, (float)_simTime);
    OnIdlePetsChanged?.Invoke(IdlePetCount, (float)_simTime);   // R160: assign leaves idle bookkeeping
    return AssignResult.Ok;
}
public UpgradeResult UpgradeJob(int jobId)
```

```
{
    if (jobId &lt; 0 || jobId &gt;= PetData.JobCount || !_jobUnlocked[jobId]) return UpgradeResult.JobLocked;
    if (_jobLevel[jobId] &gt;= PetData.JobMaxLevel) return UpgradeResult.Capped;   // R160 invest cap
    double cost = PetData.JobUpgradeCost(jobId, _jobLevel[jobId]);
    if (_coin &lt; cost) return UpgradeResult.NoCoin;
    _coin -= cost;
    _jobLevel[jobId]++;
    OnJobUpgraded?.Invoke(jobId, _jobLevel[jobId], (float)_simTime);
    return UpgradeResult.Ok;
}
/// &lt;summary&gt;Promote gate: monotonic triple threshold (jobs / dex / stars), zero lockout face.&lt;/summary
public PromoteResult TryPromote()
{
    if (_rankIndex &gt;= PetData.RankCount - 1) return PromoteResult.MaxRank;
    int g = _rankIndex; // gate from rank g to g+1
    if (JobsOpenCount &lt; PetData.GateJobsAtLeast[g] ||
        _dexCount &lt; PetData.GateDexAtLeast[g] ||
        _starLevel &lt; PetData.GateStarsAtLeast[g])
        return PromoteResult.GatesNotMet;
    _rankIndex++;
    if (FirstPromoteT &lt; 0f) FirstPromoteT = (float)_simTime;
    OnPromoted?.Invoke(_rankIndex, (float)_simTime);
    EvaluateAchievements();
    return PromoteResult.Ok;
}
// ----- T005 offline settlement (pack section 6: offline = online x 0.6, 12h wide cap) -----
/// &lt;summary&gt;Offline settle record (return-visit summary data face).&lt;/summary&gt;
public sealed class OfflineSettleResult
{
    public double AwaySec;           // raw away span fed by the caller (real-clock face)
    public double CreditedSec;       // min(away, 12h cap)
    public double OnlineRatePerSec;  // online efficiency snapshot at settle time
    public double Coin;              // credited wage coin (before the optional double)
    public double StarDust;          // credited star dust
    public int OnDuty;              // pets on duty at settle
}
/// &lt;summary&gt;
/// T005 offline settle op (data face). Offline production = online efficiency
/// x 0.6 with the 12h wide cap; both resource streams (wages + star dust)
/// settle together because the pack law prices the whole online efficiency,
/// not just the coin face. The lump credit is deterministic (a pure function
/// of the away span + the live roster), leaves the dispatch state untouched,
/// and raises one return-visit double-offer slot when there is something to
/// double (empty roster = honest zero, no hostage UX). Star-ups feed on the
/// credited star dust on the NEXT online tick (the Tick while-loop owns level
/// transitions - settling never mutates rank/dex/dispatch state). Refusals
/// (away &amp;lt;= 0) are zero-state-change verdicts.
/// &lt;/summary&gt;
public OfflineSettleResult SettleOffline(double awaySec)
{
```

```
if (awaySec &lt;= 0.0)
{
    var empty = new OfflineSettleResult { AwaySec = awaySec, CreditedSec = 0.0 };
    OnOfflineSettled?.Invoke(empty, (float)_simTime);
    return empty;   // refused, zero state change
}
double rate = 0.0;
for (int j = 0; j &lt; PetData.JobCount; j++)
{
    if (_jobUnlocked[j] &amp;&amp; _jobPet[j] &gt;= 0)
        // U204 slice 3: the settle prices the WHOLE live efficiency, so the
        // affinity match coefficient rides the away rate too (offline =
        // online x 0.6 parity law stays exact on both faces).
        rate += PetData.JobRatePerSec(j, _jobLevel[j])
                * (PetData.IsAffinityMatch(_jobPet[j], j) ? PetData.AffinityWageCoef : 1.0);
}
double creditedSec = Math.Min(awaySec, OfflineCapSec);
int onDuty = OnDutyPetCount;
double starPerMin = PetData.StarDustPerMinPerPet * onDuty
                    * (1.0 + PetData.StarDustDexBonusCoef * _dexCount);
var r = new OfflineSettleResult
{
    AwaySec = awaySec,
    CreditedSec = creditedSec,
    OnlineRatePerSec = rate,
    OnDuty = onDuty,
    Coin = rate * creditedSec * OfflineCoef,
    StarDust = starPerMin / 60.0 * creditedSec * OfflineCoef,
};
_coin += r.Coin;
_starDust += r.StarDust;
_hasDoubleOfferPending = r.Coin &gt; 0.0;   // nothing to double = no offer face
// T008: away work credits the on-duty pairs' postcard bars (true idle
// game - pack section 8 bars accumulate across away spans), grants any
// bars that fill, then journals the return-visit summary postcard
// (requirement doc M08 "the crew writes home"; zero summary = no card,
// no hostage UX). Grants are pure records - coin / star faces stay exact.
for (int j = 0; j &lt; PetData.JobCount; j++)
{
    int occ = _jobPet[j];
    if (occ &lt; 0 || !_jobUnlocked[j]) continue;
    int k = occ * PetData.JobCount + j;
    _pairWorkSec[k] += creditedSec * _ngPairBarCoef;
    TryGrantPairPostcard(occ, j);
}
if (r.Coin &gt; 0.0)
    AppendPostcard(new PostcardRecord
    {
        Seq = ++_postcardsSent,
        Kind = PetData.PostcardKind.Away,
```

```
            Pet = -1,
            Job = -1,
            WorkSec = creditedSec,
            Coin = r.Coin,
            StarDust = r.StarDust,
        });
    LastOfflineSettle = r;
    OnOfflineSettled?.Invoke(r, (float)_simTime);
    EvaluateAchievements();
    return r;
}
/// &lt;summary&gt;
/// T005 return-visit double offer claim (pack section 6/7: 1/day, x2; R73 red
/// line: every ad gain &amp;lt;= x2). Data-layer semantics: one settle raises at most
/// one claimable slot; claiming credits the base amount once more (base + base
/// = x2 total) and consumes the slot - the base is never held hostage (U091:
/// experience first, the ad is an optional bonus). T007 adds the ad gate: the
/// caller must certify the rewarded view was consumed (AdCap g15_double
/// pre-check + TryConsumeRewarded live in the SYS10 caller face); false =
/// zero-state-change refusal and the slot stays claimable. The per-day
/// frequency budget belongs to the AdCap category policy (T007 AdCapBoostCheck
/// owns HasCategoryBudget); this op owns the slot state machine only.
/// &lt;/summary&gt;
public bool TryClaimOfflineDouble(bool adConsumed = true)
{
    if (!adConsumed) return false;                        // T007 ad gate: refused, zero change
    if (!_hasDoubleOfferPending || LastOfflineSettle == null) return false;   // refused, zero change
    _hasDoubleOfferPending = false;
    double bonus = LastOfflineSettle.Coin * (OfflineDoubleMult - 1.0);
    _coin += bonus;
    _adsWatched++;   // T012: the ad certificate was honored - the zero-ad honor counts it
    OnOfflineDoubleClaimed?.Invoke(bonus, (float)_simTime);
    EvaluateAchievements();
    return true;
}
// ----- T026 sign-in retention (pack section 12; R162 check-in spectrum) -----
/// &lt;summary&gt;Wall ms per day constant (day index = wallNowMs / WallMsPerDay).&lt;/summary&gt;
public const long WallMsPerDay = 86400000L;
public sealed class SignInResult
{
    public bool Ok;
    public long WallDay;     // caller's wall day index for this claim attempt
    public int Streak;       // streak AFTER the claim (refusals mirror the live streak)
    public int CycleIndex;   // reward slot paid (0..6)
    public double Reward;    // coin credited (0 on every refusal)
}
/// &lt;summary&gt;
/// T026 sign-in claim op (data face; the View slice owns the boot popup).
/// One claim per WALL-CLOCK day fed by the caller (X995 law: uptime days
/// are blind across sessions - the caller passes wall ms, the sim derives
```

```
/// the day index, the sim never reads a clock). Streak laws (pack section
/// 12): a consecutive day increments; a gap SOFT-RESETS to day 1 - the
/// comedy zero-penalty face (a missed day never loses coins or scolds,
/// R73 red line; the streak simply starts walking the ramp again). The
/// reward walks the frozen 7-day cycle (60..300). Refusals are
/// zero-state-change verdicts: same-day re-claim, or a wall clock that
/// ran backwards (defensive - a tampered/rolled-back clock never pays).
/// &lt;/summary&gt;
public SignInResult TryClaimSignIn(long wallNowMs)
{
    long wallDay = wallNowMs / WallMsPerDay;   // floor division (epoch starts at UTC midnight)
    var res = new SignInResult { WallDay = wallDay, Streak = _signInStreak };
    if (wallDay &lt; 0 || (_signInWallDay &gt;= 0 &amp;&amp; wallDay &lt; _signInWallDay))
        return res;   // bogus feed or clock ran backwards: refused, zero state change
    if (_signInClaimedToday &amp;&amp; wallDay == _signInWallDay)
        return res;   // same-day re-claim: refused, zero state change
    if (_signInWallDay &lt; 0 || wallDay &gt; _signInWallDay + 1)
        _signInStreak = 1;   // first claim ever, or a gap: soft reset (comedy zero-penalty face)
    else
        _signInStreak++;     // consecutive day
    _signInWallDay = wallDay;
    _signInClaimedToday = true;
    _signInDoubledToday = false;   // T026-c: a fresh claim arms a fresh double latch
    res.Ok = true;
    res.Streak = _signInStreak;
    res.CycleIndex = (_signInStreak - 1) % PetData.SignInCycleDays;
    res.Reward = PetData.SignInRewards[res.CycleIndex];
    _coin += res.Reward;
    OnSignInResolved?.Invoke(_signInStreak, res.CycleIndex, res.Reward, (float)_simTime);
    return res;
}
public sealed class SignInPreview
{
    public bool Claimable;
    public int NextStreak;     // streak the next claim would land (1 after a gap)
    public int CycleDay;      // 1..7 position inside the wage cycle
    public double NextReward;  // coin the next claim would pay
}
/// &lt;summary&gt;
/// T026 slice-b pure preview of the next claim (zero writes, zero events):
/// the View badge and popup faces read this so the day-derivation and
/// streak-walk laws stay single-sourced here (the View never re-derives
/// the refusal prefix of TryClaimSignIn). Not-claimable feeds mirror the
/// refusal laws exactly: bogus/backwards feeds and the claimed-today
/// latch both return Claimable=false.
/// &lt;/summary&gt;
public SignInPreview PeekSignIn(long wallNowMs)
{
    long wallDay = wallNowMs / WallMsPerDay;
    var p = new SignInPreview();
```

```
    if (wallDay &lt; 0 || (_signInWallDay &gt;= 0 &amp;&amp; wallDay &lt; _signInWallDay)) return p;
    if (_signInClaimedToday &amp;&amp; wallDay == _signInWallDay) return p;
    p.Claimable = true;
    p.NextStreak = (_signInWallDay &lt; 0 || wallDay &gt; _signInWallDay + 1) ? 1 : _signInStreak + 1;
    p.CycleDay = ((p.NextStreak - 1) % PetData.SignInCycleDays) + 1;
    p.NextReward = PetData.SignInRewards[(p.NextStreak - 1) % PetData.SignInCycleDays];
    return p;
}
/// &lt;summary&gt;
/// T026-c sign-in x2 double op (data face; the View slice owns the ad arm).
/// The IAA 4th seat: an optional post-claim bonus that pays the day's
/// SignInRewards slot once more on the SAME wall day as the claim (claim
/// + bonus = exactly x2, R73 red line; FLAT law - no wage coefficient
/// ever stacks on the bonus). The caller must certify the rewarded view
/// was consumed (U091 ad gate; the AdCap g15_signin pre-check +
/// TryConsumeRewarded live in the SYS10 caller face); false = a
/// zero-state-change refusal. The refusal family mirrors the claim op:
/// no claim this wall day, an already-doubled day, a next-day feed (the
/// claim belongs to a previous wall day) or a clock that ran backwards
/// never pays - and the base claim is never held hostage.
/// &lt;/summary&gt;
public SignInDoubleResult TryDoubleSignIn(long wallNowMs, bool adConsumed)
{
    long wallDay = wallNowMs / WallMsPerDay;   // floor division, same domain as the claim
    var res = new SignInDoubleResult { WallDay = wallDay, Streak = _signInStreak };
    if (!adConsumed) return res;   // ad gate: refused, zero state change
    if (wallDay &lt; 0 || (_signInWallDay &gt;= 0 &amp;&amp; wallDay &lt; _signInWallDay))
        return res;   // bogus feed or clock ran backwards: refused, zero state change
    if (!_signInClaimedToday || wallDay != _signInWallDay)
        return res;   // nothing claimed this wall day (or the claim was yesterday): refused
    if (_signInDoubledToday)
        return res;   // same-day re-double: refused, zero state change (single-shot latch)
    _signInDoubledToday = true;
    res.Ok = true;
    res.CycleIndex = (_signInStreak - 1) % PetData.SignInCycleDays;
    res.Bonus = PetData.SignInRewards[res.CycleIndex];   // flat mirror of the day's slot
    _coin += res.Bonus;
    _adsWatched++;   // T012: the ad certificate was honored - the zero-ad honor counts it
    OnSignInDoubled?.Invoke(_signInStreak, res.CycleIndex, res.Bonus, (float)_simTime);
    return res;
}
public sealed class SignInDoubleResult
{
    public bool Ok;
    public long WallDay;     // caller's wall day index for this double attempt
    public int Streak;      // live streak (mirrors the claim face on refusals)
    public int CycleIndex;   // reward slot the bonus mirrors (0..6)
    public double Bonus;     // coin credited (0 on every refusal)
}
// ----- T007 IAA boosts (pack section 7; data layer - offers are a View face) -----
```

```
/// &lt;summary&gt;
/// T007 rescue boost: the peak short-staff rescue ad opens a 30-minute
/// shop-wide x2 wage window (P02-T006 rescue paradigm). The AdCap budget
/// pre-check and consume live in the caller (SYS10 HasCategoryBudget /
/// TryConsumeRewarded on g15_rescue); this op grants only when the caller
/// certifies the view was consumed - false = zero-state-change refusal.
/// A window already open refuses (the View pre-offer gate hides the seat;
/// the data layer stays honest for direct calls). Zero-RNG law intact.
/// &lt;/summary&gt;
public BoostResult TryActivateRescueBoost(bool adConsumed)
{
    if (!adConsumed) return BoostResult.AdNotConsumed;
    if (RescueBoostActive) return BoostResult.AlreadyActive;
    _rescueBoostUntil = _simTime + AdBoostDurSec;
    _adsWatched++;   // T012: granted on a consumed rewarded view - zero-ad honor counts it
    OnAdBoostActivated?.Invoke(BoostKindRescue, -1, (float)_simTime);
    EvaluateAchievements();
    return BoostResult.Ok;
}
/// &lt;summary&gt;
/// T007 speed boost: the current-job ad opens a 30-minute x2 window on one
/// occupied job (pack section 7: "current job output x2"). Same caller-side
/// AdCap contract as the rescue seat (g15_speed); an empty or locked job
/// refuses with zero state change - the ad can never be wasted on a job
/// that produces nothing (U091 honesty law). Re-aiming a live window at a
/// different job refuses while the first window runs (single-seat law).
/// &lt;/summary&gt;
public BoostResult TryActivateSpeedBoost(int jobId, bool adConsumed)
{
    if (!adConsumed) return BoostResult.AdNotConsumed;
    if (jobId &lt; 0 || jobId &gt;= PetData.JobCount || !_jobUnlocked[jobId]) return BoostResult.JobLocked;
    if (_jobPet[jobId] &lt; 0) return BoostResult.JobEmpty;
    if (_speedBoostUntil &gt; _simTime) return BoostResult.AlreadyActive;
    _speedBoostJob = jobId;
    _speedBoostUntil = _simTime + AdBoostDurSec;
    _adsWatched++;   // T012: granted on a consumed rewarded view - zero-ad honor counts it
    OnAdBoostActivated?.Invoke(BoostKindSpeed, jobId, (float)_simTime);
    EvaluateAchievements();
    return BoostResult.Ok;
}
// ----- T006 event engine (pack section 5; data layer - slice A) -----
/// &lt;summary&gt;Deterministic 32-bit mix (zero RNG law: identical inputs, identical rolls).&lt;/summary&gt;
static uint Mix32(uint x)
{
    x ^= x &gt;&gt; 16; x *= 0x7feb352du; x ^= x &gt;&gt; 15; x *= 0x846ca68bu; x ^= x &gt;&gt; 16;
    return x;
}
/// &lt;summary&gt;Cadence window roll: 180-300s slot picked deterministically per ordinal.&lt;/summary&gt;
static double CadenceSecFor(int ordinal)
{
```

```
    double span = EventCadenceMaxSec - EventCadenceMinSec;
    uint slot = Mix32((uint)ordinal) % (uint)(span + 1.0);
    return EventCadenceMinSec + slot;
}
/// &lt;summary&gt;
/// Roll the next cadence event among events whose job has a pet on duty (pack
/// law: random on-duty pet). Deterministic hash keeps the zero-RNG isomorphism
/// law intact; empty eligible set folds to -1 (honest skip, no hostage face).
/// Peak-hour weighting is a real-clock View face and lands with slice B.
/// &lt;/summary&gt;
int RollCadenceEvent()
{
    int eligible = 0;
    for (int e = 0; e &lt; PetData.EventCount; e++)
    {
        int j = PetData.Events[e].Job;
        if (_jobUnlocked[j] &amp;&amp; _jobPet[j] &gt;= 0) eligible++;
    }
    if (eligible == 0) return -1;
    uint pick = Mix32(0x9e3779b9u + (uint)_eventResolveCount) % (uint)eligible;
    for (int e = 0; e &lt; PetData.EventCount; e++)
    {
        int j = PetData.Events[e].Job;
        if (!(_jobUnlocked[j] &amp;&amp; _jobPet[j] &gt;= 0)) continue;
        if (pick == 0) return e;
        pick--;
    }
    return -1;   // unreachable
}
/// &lt;summary&gt;
/// Settle one event occurrence: star-dust correction (pack window) + album
/// collect bit (SYS08) + card data face for the View pop layer. Star level-ups
/// feed on the next star pass in Tick (the pass owns level transitions).
/// &lt;/summary&gt;
void ResolveEvent(int eventId)
{
    var def = PetData.Events[eventId];
    _starDust += def.StarDelta;
    if (def.Collect) _eventCollected[eventId] = true;
    _eventResolveCount++;
    LastEventCard = new EventCard { EventId = eventId, Id = def.Id, Fx = def.Fx, StarDelta = def.StarDelta };
    OnEventResolved?.Invoke(eventId, def.Fx, def.StarDelta, (float)_simTime);
    AdvanceChoiceWindow();
}
/// &lt;summary&gt;
/// U204: every auto resolve advances the choice window (once per 3-5
/// resolves, deterministic hash roll - R161 anti decision-fatigue). At zero
/// the next eligible choice card opens and auto cadence pauses until the
/// player settles an arm (one decision at a time). Empty eligible set folds
/// to an honest skip on a fresh window (zero hostage card).
```

```
    /// &lt;/summary&gt;
    void AdvanceChoiceWindow()
    {
        if (!PetData.ChoiceViewLive || _choicePendingId &gt;= 0) return;
        if (_choiceWindowLeft &lt; 0)
        {
            int span = PetData.ChoiceCadenceMaxResolves - PetData.ChoiceCadenceMinResolves + 1;
            _choiceWindowLeft = PetData.ChoiceCadenceMinResolves
                + (int)(Mix32(0x51ed270bu + (uint)_eventResolveCount) % (uint)span);
        }
        _choiceWindowLeft--;
        if (_choiceWindowLeft &gt; 0) return;
        int c = RollChoiceEvent();
        if (c &gt;= 0)
        {
            _choicePendingId = c;
            _choiceWindowLeft = -1;   // next window rolls fresh after the settle
            OnChoiceOpened?.Invoke(c, (float)_simTime);   // U204 slice B: the card pops on the open frame
        }
        else _choiceWindowLeft = PetData.ChoiceCadenceMinResolves;   // honest skip, re-window
    }
    /// &lt;summary&gt;
    /// U204: roll the next choice card among choice events whose job has a pet
    /// on duty AND whose story flag is satisfied (R161 callback arc - the B arm
    /// on C01 unlocks C02). Deterministic hash pick, zero RNG; empty set = -1.
    /// &lt;/summary&gt;
    int RollChoiceEvent()
    {
        int eligible = 0;
        for (int c = 0; c &lt; PetData.ChoiceCount; c++)
        {
            var d = PetData.ChoiceEvents[c];
            if (d.ReqFlag &gt;= 0 &amp;&amp; !_choiceFlags[d.ReqFlag]) continue;
            if (!_jobUnlocked[d.Job] || _jobPet[d.Job] &lt; 0) continue;
            eligible++;
        }
        if (eligible == 0) return -1;
        uint pick = Mix32(0x85ebca6bu + (uint)_eventResolveCount) % (uint)eligible;
        for (int c = 0; c &lt; PetData.ChoiceCount; c++)
        {
            var d = PetData.ChoiceEvents[c];
            if (d.ReqFlag &gt;= 0 &amp;&amp; !_choiceFlags[d.ReqFlag]) continue;
            if (!(_jobUnlocked[d.Job] &amp;&amp; _jobPet[d.Job] &gt;= 0)) continue;
            if (pick == 0) return c;
            pick--;
        }
        return -1;   // unreachable
    }
    /// &lt;summary&gt;
    /// U204: settle the open choice card (single tap, one of two arms - R161
```

```
/// gesture law: reading and deciding, not executing). Arm A = big stardust
/// now; arm B = smaller stardust + album bit + story flag (cozy
/// dual-positive: the "wrong" arm only earns less, never loses - R73). A
/// settle with no open card is a silent no-op (false, zero state change).
/// &lt;/summary&gt;
public bool ChooseEventArm(bool armA)
{
    if (_choicePendingId &lt; 0) return false;
    var def = PetData.ChoiceEvents[_choicePendingId];
    double delta = armA ? def.ChoiceAStar : def.ChoiceBStar;
    _starDust += delta;
    if (!armA)
    {
        if (def.ChoiceBCollect) _choiceCollected[_choicePendingId] = true;
        if (def.ChoiceFlag &gt;= 0) _choiceFlags[def.ChoiceFlag] = true;
    }
    _lastChoiceArmA = armA;
    // U204 FQ slice b (X1554 freeze): picking the declared risk arm plants an
    // honest wage pause on the card's source job (data face, non-hardcoded).
    if (armA &amp;&amp; def.ChoiceRiskArm)
        _jobRiskPauseUntil[def.Job] = _simTime + PetData.RiskPauseSec;
    _choiceResolveCount++;
    int settledId = _choicePendingId;
    _choicePendingId = -1;
    LastChoiceCard = new EventCard { EventId = settledId, Id = def.Id, Fx = def.Fx, StarDelta = delta };
    OnChoiceResolved?.Invoke(settledId, armA, delta, (float)_simTime);
    return true;
}
/// &lt;summary&gt;
/// Advance the idle clock: hook beat on frame zero, star dust natural accumulation
/// (no check-in, no contest), per-job wage beats on the 1s cadence while occupied.
/// &lt;/summary&gt;
public void Tick(float dt)
{
    if (dt &lt;= 0f) return;
    if (HookReadyT &lt; 0f)
    {
        HookReadyT = (float)_simTime;   // hook content lives from frame zero
        OnHookReady?.Invoke((float)_simTime);
    }
    _simTime += dt;
    if (OnDutyPetCount &gt; 0)
    {
        double perMin = PetData.StarDustPerMinPerPet * OnDutyPetCount
                        * (1.0 + PetData.StarDustDexBonusCoef * _dexCount);
        _starDust += perMin / 60.0 * dt;
        while (_starLevel &lt; PetData.StarMax &amp;&amp;
               _starDust &gt;= PetData.StarUpCostBase * (_starLevel + 1))
        {
            _starDust -= PetData.StarUpCostBase * (_starLevel + 1);
```

```
            _starLevel++;
            if (FirstStarUpT &lt; 0f) FirstStarUpT = (float)_simTime;
            OnStarUp?.Invoke(_starLevel, (float)_simTime);
        }
    }
    bool rescueOn = RescueBoostActive;   // T007: snapshot once per tick (window readouts)
    for (int j = 0; j &lt; PetData.JobCount; j++)
    {
        if (!_jobUnlocked[j] || _jobPet[j] &lt; 0) continue;   // empty job produces nothing
        _jobBeatCarry[j] += dt;
        while (_jobBeatCarry[j] &gt;= WageBeatPeriodSec)
        {
            _jobBeatCarry[j] -= WageBeatPeriodSec;
            // T007: rescue (shop-wide) and speed (single job) windows boost the
            // beat, but never stack beyond the x2 red line (pack section 7 law 1).
            bool boosted = rescueOn || SpeedBoostActive(j);
            // U204 slice 3: affinity match wage law (R160) - the occupying
            // pet's AffinityJob pays AffinityWageCoef on the beat (base-rate
            // face like the NG+ crew perk; mismatch = exact base, zero penalty).
            int occ = _jobPet[j];
            // U204 FQ slice b (X1554 freeze): wages gate on the risk-arm pause
            // window - the beat keeps firing with Amount=0 (honest zero face;
            // F5 zero-miss law: a paused beat is a registered beat, not a skip).
            double delta = _simTime &lt; _jobRiskPauseUntil[j] ? 0.0
                : PetData.JobRatePerSec(j, _jobLevel[j]) * WageBeatPeriodSec
                * (boosted ? AdBoostAmp : 1.0) * _ngWageCoef
                * (PetData.IsAffinityMatch(occ, j) ? PetData.AffinityWageCoef : 1.0);
            _coin += delta;
            WageBeatCount++;
            // T008: attribute the beat to the occupying pet's (pet, job)
            // postcard progress bar (pack section 8 - work time accumulates).
            if (occ &gt;= 0) _pairWorkSec[occ * PetData.JobCount + j] += WageBeatPeriodSec * _ngPairBarCoef;
            if (FirstWageBeatT &lt; 0f) FirstWageBeatT = (float)_simTime;
            OnCoinBeat?.Invoke((float)_simTime, delta, j);
        }
    }
    // T006: comedy event beats (pack section 5). First card = scripted E01
    // boot beat inside the &lt;=10s anchor (red line 11 first-laugh beat); later
    // beats ride the 180-300s cadence window and roll among events whose
    // job has a pet on duty. Empty eligible set = honest skip, rescheduled
    // on the same window (zero-RNG law keeps fingerprints byte-identical).
    if (!_firstEventFired &amp;&amp; _simTime &gt;= FirstEventCardAtSec)
    {
        _firstEventFired = true;
        FirstEventT = (float)_simTime;
        ResolveEvent(0);   // E01 corgi tumble: the scripted first-laugh card
        _nextEventAt = _simTime + CadenceSecFor(_eventResolveCount);
    }
    else if (_nextEventAt &gt; 0.0 &amp;&amp; _simTime &gt;= _nextEventAt)
    {
```

```
            // U204: a pending choice card pauses the auto cadence - one
            // decision at a time (reading and deciding, R161 gesture law).
            int e = _choicePendingId &gt;= 0 ? -1 : RollCadenceEvent();
            if (e &gt;= 0) ResolveEvent(e);
            _nextEventAt = _simTime + CadenceSecFor(_eventResolveCount);
        }
        // T008: wage postcard pass (pack section 8): one collectible per (pet,
        // job) pair per full 4h work bar. Progress-bar collection - bars
        // accumulate per occupied beat and never decay (zero probability,
        // guaranteed payout - R72 "letters were pure luck" counter-evidence).
        for (int j = 0; j &lt; PetData.JobCount; j++)
        {
            int occ = _jobPet[j];
            if (occ &gt;= 0 &amp;&amp; _jobUnlocked[j]) TryGrantPairPostcard(occ, j);
        }
        // T012: honor pass - wage beats, star-ups, event resolves and postcard
        // grants all land inside Tick, so the wall evaluates once per tick.
        EvaluateAchievements();
        // T013: finale latch (M10) - the ready condition can complete either on
        // a promote or on the last job opening, so the check rides the tick: it
        // fires OnEndgameReady exactly once per run, the frame the condition
        // first holds. The shown bit stays the View's latch (one ending per file).
        if (!_endgameFired &amp;&amp; EndgameReady)
        {
            _endgameFired = true;
            OnEndgameReady?.Invoke((float)_simTime);
        }
    }
    // ----- T012 achievement engine (SYS08 honor wall; data layer - the View renders) -----
    /// &lt;summary&gt;
    /// Sequential honor evaluation: every not-yet-unlocked achievement checks
    /// its condition against the live state; a met condition flips the unlock
    /// bit and fires OnAchievementUnlocked the same pass. The legend (index 7)
    /// evaluates last, so a single promote can cascade GlobalCorp -&gt; ZeroAdClear
    /// -&gt; WorkLegend within one call. Conditions are one-way (an unlock never
    /// re-locks) and every judgment maps 1:1 onto the PetData table comment.
    /// &lt;/summary&gt;
    void EvaluateAchievements()
    {
        for (int a = 0; a &lt; PetData.AchievementCount; a++)
        {
            if (_achievementUnlocked[a]) continue;
            bool met;
            switch (a)
            {
                case PetData.AchFirstWage: met = WageBeatCount &gt;= 1; break;
                case PetData.AchFullCrew: met = _dexCount &gt;= PetData.PetCount; break;
                case PetData.AchShopStarFull: met = _starLevel &gt;= PetData.StarMax; break;
                case PetData.AchPostcardCollector: met = _postcardsSent &gt;= PetData.PostcardCollectorTarget; break
                case PetData.AchEventWitness: met = EventAlbumOpenCount &gt;= PetData.EventCount; break;
```

```
                case PetData.AchGlobalCorp: met = _rankIndex &gt;= PetData.RankCount - 1; break;
                case PetData.AchZeroAdClear:
                    met = _rankIndex &gt;= PetData.RankCount - 1 &amp;&amp; _adsWatched == 0;
                    break;
                case PetData.AchWorkLegend:
                    met = true;
                    for (int o = 0; o &lt; PetData.AchWorkLegend; o++)
                        if (!_achievementUnlocked[o]) { met = false; break; }
                    break;
                default: met = false; break;
            }
            if (!met) continue;
            _achievementUnlocked[a] = true;
            _lastAchievementId = a;
            if (FirstAchievementT &lt; 0f) FirstAchievementT = (float)_simTime;
            OnAchievementUnlocked?.Invoke(a, (float)_simTime);
        }
    }
    /// &lt;summary&gt;
    /// Deterministic state fingerprint (offline/online isomorphism base): two runs
    /// fed identical (dt, op) streams must produce identical strings, byte for byte.
    /// &lt;/summary&gt;
    public string ComputeFingerprint()
    {
        var sb = new StringBuilder(192);
        sb.Append(_simTime.ToString("R", CultureInfo.InvariantCulture)).Append('|');
        sb.Append(_coin.ToString("R", CultureInfo.InvariantCulture)).Append('|');
        sb.Append(_starDust.ToString("R", CultureInfo.InvariantCulture)).Append('|');
        sb.Append(_starLevel).Append('|').Append(_rankIndex).Append('|').Append(_dexCount).Append('|');
        for (int i = 0; i &lt; PetData.PetCount; i++) sb.Append(_petRecruited[i] ? '1' : '0');
        sb.Append('|');
        for (int j = 0; j &lt; PetData.JobCount; j++)
        {
            sb.Append(_jobUnlocked[j] ? '1' : '0').Append(':').Append(_jobLevel[j]).Append(':').Append(_jobPet[j]);
            if (j &lt; PetData.JobCount - 1) sb.Append(';');
        }
        sb.Append('|').Append(_eventResolveCount).Append('|');
        for (int e = 0; e &lt; PetData.EventCount; e++) sb.Append(_eventCollected[e] ? '1' : '0');
        // U204 choice face (gate-off drives fold to constant zeros - identity)
        sb.Append('|').Append(_choiceResolveCount).Append('|').Append(_choicePendingId);
        for (int f = 0; f &lt; PetData.ChoiceFlagCount; f++) sb.Append(_choiceFlags[f] ? '1' : '0');
        for (int c = 0; c &lt; PetData.ChoiceCount; c++) sb.Append(_choiceCollected[c] ? '1' : '0');
        sb.Append('|').Append(_rescueBoostUntil.ToString("R", CultureInfo.InvariantCulture))
          .Append('|').Append(_speedBoostJob)
          .Append('|').Append(_speedBoostUntil.ToString("R", CultureInfo.InvariantCulture))
          .Append('|').Append(_postcardsSent).Append('|').Append(_postcards.Count).Append('|');
        // T008 pair album bits, packed 4-per-hex-nibble (fixed-width face, 30 chars).
        for (int k = 0; k &lt; PetData.PostcardPairCount; k += 4)
        {
            int nib = (_pairCardCollected[k] ? 8 : 0) | (_pairCardCollected[k + 1] ? 4 : 0)
```

```
                | (_pairCardCollected[k + 2] ? 2 : 0) | (_pairCardCollected[k + 3] ? 1 : 0);
            sb.Append(nib.ToString("X", CultureInfo.InvariantCulture));
        }
        // T026 sign-in face (wall day + streak + claimed bit; constant -1|0|0
        // until the first claim - an additive face, byte-stable for streams
        // that never sign in).
        sb.Append('|').Append(_signInWallDay).Append('|').Append(_signInStreak)
          .Append('|').Append(_signInClaimedToday ? '1' : '0');
        // T012 honor wall faces: unlock bits + the zero-ad condition counter.
        sb.Append('|');
        for (int a = 0; a &lt; PetData.AchievementCount; a++) sb.Append(_achievementUnlocked[a] ? '1' : '0');
        sb.Append('|').Append(_adsWatched);
        // T013 finale faces: ready latch + shown latch (one ending per file).
        sb.Append('|').Append(_endgameFired ? '1' : '0').Append(_endgameShown ? '1' : '0');
        // T013 slice C: NG+ restart ordinal (run counter across inheritance).
        sb.Append('|').Append(_ngPlusCount);
        // U204 depth slice: the cultivation lane of the current run (0=None).
        sb.Append('|').Append((int)_ngPlusPerk);
        return sb.ToString();
    }
    // ----- T008 postcard engine (pack section 8; data layer - slice B renders) -----
    /// &lt;summary&gt;Postcard journal record: the share + album data face.&lt;/summary&gt;
    public sealed class PostcardRecord
    {
        public int Seq;            // journal ordinal (1-based)
        public PetData.PostcardKind Kind;   // Wage = pair collectible; Away = return-visit summary
        public int Pet;             // -1 on the away card
        public int Job;             // -1 on the away card
        public double WorkSec;      // credited work (wage) or away (away) seconds
        public double Coin;         // away card credited coin (wage cards are work-time collectibles)
        public double StarDust;     // away card credited star dust
    }
    /// &lt;summary&gt;Grant the (pet, job) pair card when its work bar fills. Pure
    /// record: coin / star faces never move (a grant is a collection event,
    /// not a payout - the wage itself already landed beat by beat).&lt;/summary&gt;
    void TryGrantPairPostcard(int petId, int jobId)
    {
        int k = petId * PetData.JobCount + jobId;
        if (_pairCardCollected[k] || _pairWorkSec[k] &lt; PetData.PostcardWorkSecPerCard) return;
        _pairCardCollected[k] = true;
        AppendPostcard(new PostcardRecord
        {
            Seq = ++_postcardsSent,
            Kind = PetData.PostcardKind.Wage,
            Pet = petId,
            Job = jobId,
            WorkSec = _pairWorkSec[k],
        });
    }
    /// &lt;summary&gt;Journal append with the pack capacity law (FIFO eviction at
```

```
/// cap). Static + public so the gate can prove the bound without 120
/// live grants; the sim uses it through the instance wrapper below.&lt;/summary&gt;
public static void AppendPostcardBounded(List&lt;PostcardRecord&gt; journal, PostcardRecord rec, int capacity)
{
    if (capacity &gt; 0 &amp;&amp; journal.Count &gt;= capacity) journal.RemoveAt(0);
    journal.Add(rec);
}
void AppendPostcard(PostcardRecord rec)
{
    AppendPostcardBounded(_postcards, rec, PetData.PostcardAlbumCapacity);
    OnPostcardSent?.Invoke(rec, (float)_simTime);
}
// ----- T008 share dual channel (red line 12 / X032 law: offline share
// form - a compact code string the receiver imports and re-derives
// locally, zero server, zero networking). Channel A (the rendered card
// image) is a View face and lands with slice B; this is channel B, the
// data face both channels feed from. Cent resolution on the away money
// fields keeps the roundtrip byte-exact (the code carries the summary
// face, not the raw doubles). -----
const string ShareCodePrefix = "G15PC1";   // game tag + code schema version
static string ToB36(long v)
{
    if (v == 0) return "0";
    bool neg = v &lt; 0;
    if (neg) v = -v;
    const string alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    var sb = new StringBuilder(12);
    while (v &gt; 0) { sb.Append(alphabet[(int)(v % 36)]); v /= 36; }
    if (neg) sb.Append('-');
    char[] c = sb.ToString().ToCharArray();
    Array.Reverse(c);
    return new string(c);
}
static bool TryFromB36(string s, out long v)
{
    v = 0;
    if (string.IsNullOrEmpty(s)) return false;
    foreach (char ch in s)
    {
        int d = ch &gt;= '0' &amp;&amp; ch &lt;= '9' ? ch - '0'
              : ch &gt;= 'A' &amp;&amp; ch &lt;= 'Z' ? ch - 'A' + 10 : -1;
        if (d &lt; 0) return false;
        v = v * 36 + d;
    }
    return true;
}
static uint Fnv1a(string s)
{
    uint h = 2166136261u;
    for (int i = 0; i &lt; s.Length; i++) { h ^= (uint)s[i]; h *= 16777619u; }
```

```
            return h;
        }
        /// &lt;summary&gt;Channel B encoder: wage/away card -&gt; compact offline share code.&lt;/summary&gt;
        public static string EncodePostcardCode(PostcardRecord rec)
        {
            long coinCents = (long)Math.Round(rec.Coin * 100.0, MidpointRounding.AwayFromZero);
            long starCents = (long)Math.Round(rec.StarDust * 100.0, MidpointRounding.AwayFromZero);
            long workSec = (long)Math.Round(rec.WorkSec, MidpointRounding.AwayFromZero);
            string payload = ShareCodePrefix + "."
                + (rec.Kind == PetData.PostcardKind.Wage ? 'W' : 'A') + "."
                + ToB36(rec.Seq) + "." + ToB36(rec.Pet + 1) + "." + ToB36(rec.Job + 1) + "."
                + ToB36(workSec) + "." + ToB36(coinCents) + "." + ToB36(starCents);
            return payload + "." + ToB36((long)Fnv1a(payload));
        }
        /// &lt;summary&gt;Channel B decoder: validates the checksum, then re-derives the
        /// card summary locally. Every refusal returns false with a null record
        /// (tampered / malformed codes never touch sim state).&lt;/summary&gt;
        public static bool TryParsePostcardCode(string code, out PostcardRecord rec)
        {
            rec = null;
            if (string.IsNullOrEmpty(code)) return false;
            int last = code.LastIndexOf('.');
            if (last &lt;= 0 || last == code.Length - 1) return false;
            string payload = code.Substring(0, last);
            string check = code.Substring(last + 1);
            long want;
            if (!TryFromB36(check, out want) || want != (long)Fnv1a(payload)) return false;
            string[] f = payload.Split('.');
            if (f.Length != 8 || f[0] != ShareCodePrefix) return false;
            long seq, pet1, job1, workSec, coinCents, starCents;
            if (!TryFromB36(f[2], out seq) || !TryFromB36(f[3], out pet1) || !TryFromB36(f[4], out job1)
                || !TryFromB36(f[5], out workSec) || !TryFromB36(f[6], out coinCents) || !TryFromB36(f[7], out starCents
                return false;
            if ((f[1] != "W" &amp;amp;&amp;amp; f[1] != "A") || pet1 &lt; 0 || pet1 &gt; PetData.PetCount
                || job1 &lt; 0 || job1 &gt; PetData.JobCount || seq &lt; 1)
                return false;
            rec = new PostcardRecord
            {
                Seq = (int)seq,
                Kind = f[1] == "W" ? PetData.PostcardKind.Wage : PetData.PostcardKind.Away,
                Pet = (int)(pet1 - 1),
                Job = (int)(job1 - 1),
                WorkSec = workSec,
                Coin = coinCents / 100.0,
                StarDust = starCents / 100.0,
            };
            return true;
        }
    }
}
```